

**ANALISIS DAMPAK PANJANG DIMENSI EMBEDDING
MATRYOSHKA TERHADAP KUALITAS HASIL DAN WAKTU
EKSEKUSI DALAM SISTEM PENCARIAN PAKAR**

SKRIPSI

Disusun oleh:
Moch. Gustav Ali Rafsandjani
NIM: 225150201111015



TEKNIK INFORMATIKA
DEPARTEMEN TEKNIK INFORMATIKA
FAKULTAS ILMU KOMPUTER
UNIVERSITAS BRAWIJAYA
MALANG
2025

DAFTAR ISI

DAFTAR ISI	ii
DAFTAR TABEL	v
DAFTAR GAMBAR	vi
BAB 1 PENDAHULUAN	1
1.1 Latar Belakang	1
1.2 Rumusan Masalah	2
1.3 Tujuan	2
1.4 Manfaat	2
1.5 Batasan Masalah.....	2
1.6 Sistematika Pembahasan	2
BAB 2 LANDASAN KEPUSTAKAAN	4
2.1 Kajian Pustaka.....	4
2.2 Dasar Teori.....	6
2.2.1 Sistem Pencarian Pakar	6
2.2.2 <i>Vector Space Model</i>	7
2.2.3 <i>Matryoshka Representation Learning</i>	7
2.2.4 Pemingkatan dalam Sistem Pencarian Pakar.....	8
2.2.5 Alokasi Kredit Kepengarangan	9
2.2.6 Arsitektur Pencarian Berjenjang Dua Tahap	10
2.2.7 Metode Evaluasi Berbasis Model Bahasa Besar sebagai Juri.....	10
2.2.8 Generasi Kueri Sintetis	11
2.2.9 Sistem Penilaian ELO	11
BAB 3 METODOLOGI	13
3.1 Tipe Penelitian	13
3.2 Pengumpulan Data	13
3.3 Metode <i>Preprocessing</i>	14
3.4 Metode Evaluasi	14
3.5 Metode Analisis Data.....	15
BAB 4 PERANCANGAN	16

4.1 Deskripsi Umum	16
4.2 Arsitektur Sistem	17
4.2.1 Komponen Utama Sistem	17
4.2.2 Skema Basis Data	18
4.2.3 Interaksi Antar Komponen	19
4.3 Alur Kerja Sistem	20
4.3.1 <i>Data Ingestion</i>	20
4.3.2 Alur Vektorisasi Dokumen	24
4.3.3 Alur Pencarian Pakar	27
4.3.4 Alur Evaluasi <i>Pairwise</i>	31
4.4 <i>Expertise Modeling</i>	32
4.5 Perhitungan Manual	32
4.5.1 Persiapan Input	32
4.5.2 <i>Shortlisting</i>	34
4.5.3 <i>Reranking</i>	34
4.5.4 Agregasi Skor Kepakaran	35
BAB 5 IMPLEMENTASI	37
5.1 Deskripsi Umum Implementasi Sistem	37
5.2 Implementasi Basis Data dan <i>Pre-processing</i> Data	37
5.2.1 Implementasi <i>Ingestion</i> dan <i>Pre-processing</i> Data Scopus	37
5.2.2 Implementasi Fungsi <i>Vector Slicing</i>	39
5.3 Implementasi Pencarian Pakar	40
5.3.1 Implementasi Algoritme <i>Two-Stage Retrieval</i>	40
5.3.2 Implementasi Pemodelan dan Agregasi Skor Kepakaran	41
5.4 Implementasi Layanan Web	42
5.4.1 Implementasi API <i>Backend</i>	42
5.4.2 Implementasi Pencatatan Preferensi	44
5.4.3 Implementasi Antarmuka Pengguna	45
5.5 Implementasi <i>Pipeline</i> Evaluasi Otomatis	46
5.5.1 Implementasi Generator Kueri Sintetis	46
5.5.2 Implementasi Juri LLM	47
5.5.3 Implementasi Pencatatan Preferensi Juri LLM	49

BAB 6 HASIL DAN PEMBAHASAN	50
6.1 Hasil Eksperimen	50
6.1.1 Hasil Pengujian Latensi Vektor	50
6.1.2	51
6.1.3 S	Error! Bookmark not defined.
6.1.4 S	Error! Bookmark not defined.
6.2 Analisa Hasil.....	54
6.2.1 D.....	Error! Bookmark not defined.
6.2.2 D.....	Error! Bookmark not defined.
6.2.3 D.....	Error! Bookmark not defined.
BAB 7 PENUTUP	Error! Bookmark not defined.
DAFTAR REFERENSI	57

DAFTAR TABEL

Tabel 2.1 Celah Penelitian dalam Sistem Pencarian Pakar	4
Tabel 2.2 Estimator Kepakaran	4
Tabel 2.3 Model Pembandingan dalam Penelitian ExpFinder	5
Tabel 2.4 Teknik <i>Data Fusion</i>	9
Tabel 4.1 Contoh Representasi Vektor Kueri Setelah Penambahan Prefix	33
Tabel 4.2 Contoh Representasi Vektor Dokumen dari Basis Data	33
Tabel 4.3 Contoh Vektor Terpotong dan Ternormalisasi	34
Tabel 4.4 Hasil <i>Shortlisting</i>	34
Tabel 4.5 Hasil <i>Reranking</i>	35
Tabel 4.6 Skema <i>Author Order Weighting</i>	35
Tabel 4.7 Perhitungan <i>Weighted Score</i>	35
Tabel 4.8 Agregasi dan Pemeringkatan Akhir Skor Kepakaran	36
Tabel 6.1 sss	50

DAFTAR GAMBAR

Gambar 2.1 Komponen Utama dalam Pencarian Pakar	6
Gambar 4.1 Arsitektur Sistem Pencarian Pakar	17
Gambar 4.2 <i>Relational Schema</i> Basis Data Sistem Pencarian Pakar	19
Gambar 4.3 Diagram Alur <i>Data Ingestion</i>	21
Gambar 4.4 Diagram Alur <i>Data Ingestion</i>	22
Gambar 4.5 Diagram Alur <i>Data Ingestion</i>	23
Gambar 4.6 Diagram Alur Vektorisasi Dokumen	24
Gambar 4.7 Diagram Alur Vektorisasi Dokumen	25
Gambar 4.8 Diagram Alur Vektorisasi Dokumen	26
Gambar 4.9 Diagram Alur Pencarian Pakar	28
Gambar 4.10 Diagram Alur Pencarian Pakar	29
Gambar 4.11 Diagram Alur Pencarian Pakar	30
Gambar 4.12 Alur Evaluasi Pairwise	31

BAB 1 PENDAHULUAN

1.1 Latar Belakang

Large Language Model (LLM) menyediakan alternatif interaksi dalam pencarian informasi secara efisien (IBM, 2025). Pada tahun 2025, ChatGPT tercatat memproses lebih dari 1 miliar kueri setiap hari dari 800 juta pengguna aktif mingguan, menjadikannya salah satu platform kecerdasan buatan dengan volume penggunaan tertinggi secara global (Singh, 2025). Meskipun demikian, LLM memiliki keterbatasan inheren. Model tersebut dapat menyajikan informasi tidak akurat sebagai fakta, sebuah fenomena diklasifikasikan sebagai halusinasi (Huang et al., 2025). Selain itu, LLM menunjukkan kecenderungan *sycophancy*, yaitu kecenderungan menghasilkan respons yang disesuaikan dengan preferensi pengguna tanpa mempertimbangkan akurasi factual demi memperoleh penilaian positif (Malmqvist, 2025). Keterbatasan tersebut menyulitkan verifikasi kebenaran informasi secara mandiri dan memerlukan interpretasi tambahan dari pihak yang memiliki kepakaran (Huang et al., 2025).

Kepakaran menyediakan landasan pengetahuan dan pengalaman yang diperlukan untuk menyaring serta menginterpretasikan informasi secara tepat. Sebagai contoh, pada bidang medis, konsultasi dengan spesialis diperlukan untuk memastikan diagnosis dan perencanaan perawatan yang akurat (Bahadori et al., 2020). Keterlibatan pakar dapat meningkatkan kualitas keputusan dan meminimalkan risiko akibat kesalahan interpretasi data, yang jika diabaikan dapat menghasilkan konsekuensi negatif yang signifikan (Lighthall and Vazquez-Guillamet, 2015).

Sistem pencarian pakar diklasifikasikan ke dalam beberapa pendekatan utama. Salah satunya adalah *Generative Probabilistic Models* atau *Document-based Models*. Pada pendekatan ini, pencarian diawali dengan mengidentifikasi dokumen yang relevan dengan kueri pengguna, kemudian peringkat kandidat pakar dihitung berdasarkan tingkat asosiasi mereka dengan dokumen-dokumen tersebut (Husain et al., 2019). Pendekatan kedua adalah *Graph-based Models*, yang merepresentasikan hubungan antar pakar, seperti kolaborasi atau sitasi, dalam bentuk graf. Metrik seperti *PageRank* atau *HITS* dimanfaatkan untuk mengukur otoritas dan pengaruh seorang pakar dalam komunitasnya berdasarkan struktur jaringan, bukan sekadar konten tekstual (Kang et al., 2023). Selain itu, terdapat model *hybrid* yang mengintegrasikan berbagai sumber bukti, seperti konten tekstual, metrik sitasi, dan informasi profil, melalui Teknik agregasi peringkat seperti *CombSUM* dan *CombMNZ* (Moreira, Martins and Calado, 2020).

Kinerja sistem *Document-based Models* dibatasi oleh metode representasi semantik kepakaran yang diekstraksi dari dokumen dan kueri pengguna. Metode vektorisasi seperti TF-IDF, Word2Vec, dan BERT menghasilkan vektor berdimensi tetap. Dimensi yang tinggi meningkatkan kualitas representasi semantik tetapi membutuhkan biaya komputasi yang besar, sedangkan dimensi yang rendah beresiko menghilangkan kelengkapan informasi semantik (*rich semantic*

information). Matryoshka Representation Learning (MRL) memitigasi keterbatasan tersebut dengan menghasilkan satu embedding yang dapat diekstraksi pada berbagai panjang dimensi tanpa memerlukan pelatihan ulang (Kusupati et al., 2022). Representasi ini memungkinkan adanya kompromi (*trade-off*) antara efisiensi komputasi dan akurasi representasi. Namun, bagaimana fleksibilitas dimensi ini memengaruhi kualitas hasil pencarian dalam sistem pencarian pakar belum dieksplorasi. Oleh karena itu, penelitian ini pada pengembangan sistem pencarian pakar *Document-based Model* berbasis MRL serta analisis dampaknya terhadap preferensi pengguna.

1.2 Rumusan Masalah

1. Bagaimana pengaruh variasi panjang dimensi embedding Matryoshka terhadap waktu pencarian vektor pada sistem pencarian pakar?
2. Bagaimana dampak variasi panjang dimensi embedding Matryoshka terhadap kualitas hasil pencarian pakar?

1.3 Tujuan

1. Mengembangkan sistem pencarian pakar berbasis *Document-based Model* menggunakan MRL untuk representasi kepakaran.
2. Menganalisis pengaruh variasi panjang dimensi *embedding* Matryoshka terhadap waktu pencarian vektor dalam memproses kueri pemeringkatan pakar.

1.4 Manfaat

1. Memberikan hasil empiris bagi pengembang untuk merancang sistem pencarian pakar yang optimal dengan menyeimbangkan efisiensi komputasi dan kualitas hasil pencarian menggunakan MRL.
2. Membuka potensi penghematan biaya komputasi bagi organisasi dan industri dengan memungkinkan implementasi sistem pencarian pakar berskala besar yang efisien.

1.5 Batasan Masalah

1. Sumber data yang digunakan terbatas pada judul, abstrak, dan metadata artikel ilmiah yang terindeks dalam basis data Scopus.
2. Cakupan artikel dibatasi pada publikasi rentang tahun 2020 hingga 2025 yang terafiliasi dengan 10 institusi dengan produktivitas publikasi tertinggi di Indonesia.

1.6 Sistematika Pembahasan

Laporan penelitian ini disusun dalam tujuh bab sebagai berikut:

BAB 1 PENDAHULUAN

Memuat latar belakang masalah, tujuan, manfaat, Batasan masalah, dan sistematika pembahasan.

BAB 2 TINJAUAN PUSTAKA DAN LANDASAN TEORI

Memuat landasan konseptual mengenai sistem pencarian pakar, representasi semantik, *Matryoshka Representation Learning* (MRL), teknik pemeringkatan pakar, serta sistem penilaian Elo yang menjadi dasar evaluasi penelitian ini.

BAB 3 METODOLOGI PENELITIAN

Memuat tahapan sistematis penelitian, meliputi pengumpulan data, desain eksperimen, dan prosedur evaluasi sistem.

BAB 4 PERANCANGAN SISTEM

Memuat rancangan arsitektur dan alur kerja logis dari sistem pencarian pakar berbasis MRL yang akan dikembangkan.

BAB 5 IMPLEMENTASI DAN PENGUJIAN

Memuat rincian teknis pengembangan sistem berdasarkan perancangan dan prosedur pengujian yang digunakan.

BAB 6 HASIL DAN PEMBAHASAN

Memuat penyajian data hasil eksperimen serta analisis pengaruh variasi dimensi *embedding* terhadap waktu eksekusi dan preferensi pengguna.

BAB 7 PENUTUP

Memuat kesimpulan dari hasil penelitian serta saran untuk pengembangan sistem selanjutnya.

BAB 2 LANDASAN KEPUSTAKAAN

2.1 Kajian Pustaka

Terdapat beberapa penelitian yang dilakukan sebelumnya terkait sistem pencarian pakar. Penelitian oleh Husain dkk. (2019) menyajikan tinjauan sistematis komprehensif tentang sistem pencarian pakar dan studi pencarian kepakaran yang dipublikasikan antara tahun 2010 dan 2019. Studi ini menganalisis domain sistem pencarian pakar, sumber kepakaran, metode, dan dataset yang digunakan, serta mengidentifikasi perbedaan antara *Expert Retrieval* dan *Expert Seeking*. Hasil menunjukkan bahwa sekitar 65% sistem pencarian pakar digunakan di domain akademik. Studi ini juga mengidentifikasi celah penelitian yang menjadi arahan riset di masa depan, yang dirangkum dalam tabel 2.1.

Tabel 2.1 Celah Penelitian dalam Sistem Pencarian Pakar

No	Celah Penelitian	Deskripsi
1	Domain Aplikasi	Penelitian didominasi oleh domain akademik; domain lain seperti medis atau industri kurang mendapat fokus penelitian.
2	Ketersediaan Sumber Kepakaran	Tantangan terkait privasi data, integrasi sumber, kebaruan dan kelengkapan informasi.
3	Metode Sistem	Kebutuhan akan algoritma yang lebih canggih untuk mengintegrasikan berbagai bukti kepakaran secara efektif.
4	Dataset	Diperlukan dataset <i>benchmark</i> yang menggabungkan informasi dari berbagai sumber.
5	Kombinasi Pengambilan dan Pencarian	Kebutuhan integrasi faktor kontekstual dari pencarian kepakaran (<i>expertise seeking</i>) ke dalam sistem pengambilan (<i>expertise retrieval</i>) untuk meningkatkan efektivitas.

Sumber: Husain dkk. (2019)

Penelitian oleh Moreira, Martins dan Calado. (2020) mengeksplorasi penggunaan metode agregasi peringkat tanpa supervisi untuk menggabungkan beberapa estimator kepakaran. Eksperimen dilakukan pada dataset publikasi akademik di bidang Ilmu Komputer dan menunjukkan bahwa metode agregasi peringkat, seperti CombSUM dan CombMNZ, memberikan hasil yang kompetitif. Estimator yang digunakan berasal dari tiga sumber utama, sebagaimana dirangkum pada Tabel 2.2.

Tabel 2.2 Estimator Kepakaran

No	Jenis Fitur	Deskripsi
1	Fitur Tekstual	Berasal dari kemiripan tekstual antara kueri dan konten dokumen menggunakan skor TF-IDF atau BM25.

2	Fitur Profil	Berdasarkan informasi profil penulis yang independen dari kueri, seperti jumlah publikasi di jurnal atau konferensi.
3	Fitur Jaringan	Dihitung dari jaringan ilmiah seperti co-citation dan co-authorship, menggunakan metrik seperti PageRank dan h-index.

Sumber: Moreira, Martins dan Calado. (2020)

Penelitian oleh (Kang et al., 2023) mengusulkan ExpFinder, sebuah model ensemble untuk pencarian pakar yang mengintegrasikan model *N-gram Vector Space Model* (nVSM) dengan model berbasis graf μ CO-HITS. ExpFinder diuji pada empat dataset akademik, yaitu LEXR dan tiga dataset dari DBLP. Hasil evaluasi menunjukkan bahwa ExpFinder efektif, mengungguli semua model yang dibandingkan dengan peningkatan kinerja antara 19% hingga 160.2%. Model pembanding yang digunakan dalam penelitian ini dirangkum dalam tabel 2.3.

Tabel 2.3 Model Pembanding dalam Penelitian ExpFinder

No	Kategori Model	Deskripsi
1	<i>Vector Space Model</i> (VSM)	TF-IDF based VSM.
2	<i>Document Language Model</i> (DLM)	Model DLM oleh Balog dkk. (2012), Model WISER oleh Cifariello, Ferragina dan Ponza. (2019).
3	<i>Graph-based Model</i> (GM)	Model ADT oleh Gollapalli, Mitra dan Giles. (2013), Model Reputation oleh Schall. (2015), Topic-sensitive CO-HITS oleh Deng, Lyu dan King. (2009).

Sumber: Kang dkk. (2023)

Penelitian oleh Rampisela dan Yulianti. (2020) berfokus pada pencarian pakar akademik menggunakan word embedding dan document embedding. Metode yang digunakan adalah kombinasi Word2Vec dan Doc2Vec untuk merepresentasikan kueri dan kepakaran dosen. Hasil penelitian menunjukkan bahwa representasi kueri menggunakan Word2Vec menghasilkan kinerja yang lebih baik, dan kinerja model embedding tersebut sebanding dengan model retrieval standar BM25 dalam memproses kueri berbahasa Indonesia dan Inggris.

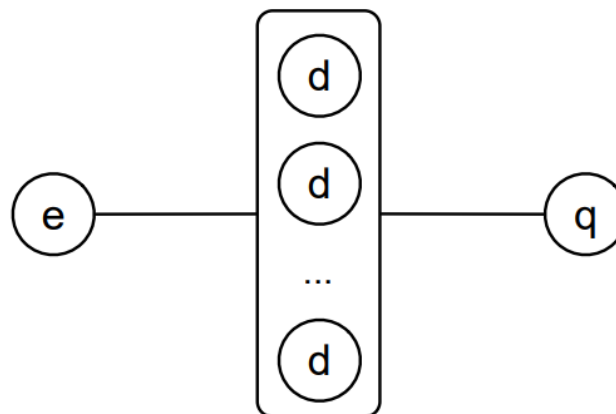
Penelitian ini memiliki beberapa kesamaan konsep dasar dengan penelitian sebelumnya. Pertama, penelitian ini mengimplementasikan sistem pencarian pakar Document-based Model, serupa dengan pendekatan yang dijelaskan oleh (Husain et al., 2019; Rampisela and Yulianti, 2020a), dengan kepakaran diekstraksi langsung dari dokumen yang diasosiasikan dengan pakar. Kedua, penelitian ini menggunakan konsep representasi semantik untuk kepakaran, yang merupakan elemen kunci dalam studi oleh (Rampisela and Yulianti, 2020b; Kang et al., 2023)

2.2 Dasar Teori

2.2.1 Sistem Pencarian Pakar

Menurut buku Expertise Retrieval oleh Balog dkk. (2012, hlm.131), sistem pencarian pakar didefinisikan sebagai area umum yang bertujuan untuk menghubungkan pakar dengan bidang kepakaran mereka, dan sebaliknya, atau lebih formal, untuk menemukan pakar berdasarkan suatu topik dan membuat profil topik kepakaran seseorang. Tujuan utamanya adalah untuk mengotomatisasi proses menemukan kepakaran, yang secara spesifik menjawab kebutuhan informasi seperti “Temukan pakar tentang topik X” (expert finding) atau “Jelaskan topik kepakaran orang Y” (expert profiling). Penelitian dalam bidang ini berupaya untuk menilai dan mengakses “pengetahuan tacit” yang sulit diformalkan dengan cara menganalisis artefak dari “pengetahuan eksplisit” seperti dokumen, email, dan publikasi ilmiah.

Sistem pencarian pakar melibatkan tiga komponen utama: pakar (experts), topik (topics), dan dokumen (documents), yang berinteraksi untuk membangun hubungan antara pakar dan topik yang dicari. Komponen Utama dalam Pencarian Pakar digambarkan pada Gambar 2.1. Sistem pencarian pakar bertujuan untuk membangun hubungan antara seorang pakar (e), suatu topik kueri (q), berdasarkan bukti yang ditemukan dalam sekumpulan dokumen (d).



Gambar 2.1 Komponen Utama dalam Pencarian Pakar

Sumber: Balog dkk. (2012, hlm.159)

Pengembangan sistem pencarian pakar dihadapkan pada beberapa tantangan signifikan. Tantangan-tantangan ini meliputi: (1) kandidat pakar tidak direpresentasikan sebagai unit yang dapat dicari secara langsung (*retrievable units*); (2) nama pakar seringkali bersifat ambigu; (3) bukti kepakaran berasal dari sumber data yang heterogen dengan tingkat kepentingan yang berbeda; dan (4) kompleksitas dalam menentukan kekuatan asosiasi antara seorang kandidat dan bukti tekstual kepakarannya. (Balog et al., 2012, pp.131–132)

Untuk mengatasi tantangan tersebut, penelitian di bidang pencarian pakar telah mengembangkan beberapa pendekatan utama. Menurut Balog dkk. (2012, hlm.137, 159), dua pendekatan probabilistik generatif yang paling fundamental adalah *profile-based model* atau *candidate model* dan *document-based model*. Pendekatan *profile-based* dilakukan dengan terlebih dahulu menggabungkan semua dokumen yang berasosiasi dengan seorang kandidat untuk membentuk sebuah profil atau *pseudo-document*. Profil ini merupakan agregasi dari semua konten tekstual yang berasosiasi dengan seorang pakar. Profil tersebut yang akan dibandingkan dengan kueri pengguna. Pendekatan *document-based model* bekerja dengan mengidentifikasi dokumen-dokumen yang relevan dengan kueri pengguna. Balog dkk. (2012, hlm. 137, 169) mencatat bahwa *document-based model* umumnya lebih efektif, lebih kuat (*robust*), dan dijadikan sebagai baseline dalam berbagai penelitian.

2.2.2 Vector Space Model

Vector Space Model (VSM) adalah sebuah model aljabar yang digunakan untuk merepresentasikan dokumen teks dan kueri sebagai vektor numerik dalam sebuah ruang multidimensi (Berry, Drmac and Jessup, 1999). Dalam model ini, setiap dimensi dalam ruang vektor mewakili sebuah kata unik dari keseluruhan kosakata. Metode pembobotan seperti *Term Frequency-Inverse Document Frequency* (TF-IDF) digunakan untuk menghitung nilai setiap dimensi, yang menghasilkan representasi vektor yang renggang (*sparse*) yang sebagian besar nilainya adalah nol. Representasi berdimensi tinggi dan renggang ini menghadapi tantangan yang dikenal sebagai *curse of dimensionality*, yaitu sebuah kondisi data menjadi terlalu jarang di dalam ruang vektor yang besar, sehingga menyulitkan model untuk melakukan inferensi atau generalisasi yang efektif (Bengio et al., 2003)

Untuk mengatasi *curse of dimensionality*, Bengio dkk. (2003) memelopori pendekatan untuk mempelajari *distributed representation* bagi setiap kata. Ide utamanya adalah mengasosiasikan setiap kata dengan sebuah vektor fitur yang padat (*dense*) dan berdimensi jauh lebih rendah yang mampu menangkap karakteristik semantik dari kata tersebut. Konsep fundamental ini menjadi landasan bagi berbagai metode *word embedding*, seperti Word2Vec yang mempelajari vektor berdasarkan konteks kata-kata di sekitarnya, dan GloVe yang memanfaatkan statistik ko-okurensi kata global dari keseluruhan korpus (Mikolov et al., 2013; Pennington, Socher and Manning, 2014). Evolusi terkini dari representasi ini adalah model berbasis arsitektur Transformer seperti BERT, yang mampu menghasilkan embedding kontekstual, yaitu representasi vektor sebuah kata bersifat dinamis dan dapat berubah sesuai dengan makna penggunaannya dalam kalimat (Devlin et al., 2019).

2.2.3 Matryoshka Representation Learning

Matryoshka Representation Learning (MRL) adalah sebuah paradigma pembelajaran representasi vektor yang bertujuan untuk mengatasi masalah

kekakuan dimensi pada representasi yang dihasilkan oleh model bahasa besar (LLM) atau model representasi modern lainnya (Kusupati et al., 2022). Secara tradisional, model seperti BERT menghasilkan vektor embedding dengan dimensi tetap (misalnya, 768 atau 1024), yang harus digunakan secara utuh terlepas dari batasan komputasi atau kebutuhan akurasi *downstream task*. Kekakuan ini sering memaksa penggunaan vektor berdimensi tinggi yang memerlukan biaya komputasi besar, terutama dalam sistem skala besar seperti sistem pencarian.

MRL memperkenalkan konsep nested vectors (vektor bersarang) yang diilhami dari boneka Matryoshka Rusia. Untuk sebuah representasi vektor berdimensi penuh $z \in \mathbb{R}^d$, MRL mengemas informasi pada berbagai granularitas. Notasikan m dimensi pertama dari vector z sebagai $z_{1:m} \in \mathbb{R}^d$, dengan $m < d$. Dalam MRL, himpunan dimensi bersarang yang dioptimalkan, $M = \{m_1, m_2, \dots, m_k\}$ telah ditentukan sebelumnya, dengan $m_1 < m_2 < \dots < m_k = d$ merupakan subset dari dimensi yang dipertimbangkan (Kusupati et al., 2022).

MRL dilatih dengan menghitung loss secara terpisah untuk setiap sub-vektor $z_{1:m}$. Fungsi loss pada setiap sub-vektor diaggregasi menjadi loss gabungan untuk pelatihan MRL. L_{Total} merupakan penjumlahan berbobot dari loss yang dihitung pada setiap level dimensi dalam himpunan M .

$$L_{Total} = \sum_{m \in M} C_m \cdot L(z_{1:m}) \quad (2.1)$$

$L(z_{1:m})$ adalah loss yang dihitung hanya menggunakan m dimensi pertama dari *embedding* dan C_m adalah skalar bobot yang menentukan kepentingan dari setiap level.

2.2.4 Pemeringkatan dalam Sistem Pencarian Pakar

Pemeringkatan adalah proses yang menghasilkan daftar kandidat pakar yang terurut yang urutan teratasnya diharapkan paling relevan dengan kueri pengguna. Mekanisme pemeringkatan ini bergantung pada model konseptual yang digunakan untuk merepresentasikan kepakaran.

Proses pemeringkatan pada *document-based model* dilakukan dalam dua tahap. Pertama, sistem terlebih dahulu mencari dan memeringkatkan dokumen yang paling relevan dengan kueri pengguna menggunakan teknik pencarian informasi standar. Skor relevansi dokumen dapat dihitung menggunakan metode seperti *cosine similarity* pada representasi vektor semantik. Kedua, skor relevansi dari dokumen-dokumen tersebut diaggregasikan untuk menghitung skor akhir setiap kandidat pakar. Setiap dokumen yang relevan memberikan suara (*vote*) kepada pakar yang berasosiasi dengannya. Bobot suara tersebut sebanding dengan tingkat relevansi dokumen terhadap kueri. Pakar yang sering muncul dalam dokumen-dokumen yang relevan akan mendapatkan skor agregat yang tinggi dan menempati peringkat teratas (Balog dkk., 2012, hlm. 169). Balog dkk. (2012, hlm.181–182) mengadaptasi beberapa teknik *data fusion* sebagai metode voting, yang dirangkum dalam tabel berikut.

Tabel 2.4 Teknik *Data Fusion*

No	Metode	Deskripsi Perhitungan Skor
1	Votes	Jumlah total dokumen relevan yang menyebutkan nama kandidat pakar.
2	RR (Reciprocal Rank)	Jumlah dari kebalikan peringkat ($1/\text{rank}$) untuk setiap dokumen relevan yang terkait dengan pakar. Metode ini memberikan bobot lebih pada dokumen yang berada di peringkat atas.
3	CombSUM	Jumlah skor relevansi dari semua dokumen relevan yang terkait dengan pakar.
4	CombMNZ	Hasil perkalian antara skor CombSUM dengan jumlah total dokumen relevan (Votes), memberikan penekanan pada pakar yang sering disebut dan dalam dokumen yang relevan.
5	expCombSUM	Varian dari CombSUM yang menggunakan transformasi eksponensial pada skor relevansi dokumen untuk memberikan bobot yang jauh lebih besar pada dokumen peringkat teratas.

Sumber: Balog dkk. (2012, hlm.181–182)

Pemeringkatan pada *profile-based model* setara dengan masalah pencarian dokumen standar dengan memperlakukan setiap profil pakar atau *pseudo-document* sebagai satu dokumen. Skor peringkat dihitung berdasarkan kemiripan langsung antara representasi semantik kueri dengan representasi semantik dari profil setiap pakar (Balog dkk., 2012, hlm. 165). Dengan demikian, pemeringkatan tidak lagi bergantung pada agregasi skor dari banyak dokumen secara *real-time*, melainkan perbandingan langsung dengan profil yang telah dibangun sebelumnya.

2.2.5 Alokasi Kredit Kepengarangan

Dalam analisis bibliometrik dan sistem pencarian pakar berbasis dokumen, pembagian kontribusi penulis pada publikasi kolaboratif merupakan aspek krusial. Pendekatan konvensional sering kali memperlakukan semua penulis secara setara atau membagi kredit secara penuh kepada setiap rekan penulis. Tindakan tersebut berpotensi menimbulkan bias ekualisasi dan inflasi yang dapat mendistorsi hasil pemeringkatan kepakaran (Hagen, 2010).

Guna merepresentasikan kontribusi riil, sistem pencarian dapat menerapkan prinsip *Sequence-Determines-Credit* (SDC). Melalui prinsip ini, urutan kepengarangan digunakan sebagai indikator utama porsi kontribusi intelektual peneliti (Hagen, 2010). Salah satu metode SDC yang diakui secara akademis adalah *Harmonic Counting*, yaitu metode yang mendistribusikan kredit berdasarkan deret harmonik bagi setiap urutan penulis.

Meskipun *Harmonic Counting* terbukti adil secara matematis, metode tersebut memiliki kelemahan karena kredit untuk penulis pada urutan akhir menurun

secara eksponensial. Pada konvensi publikasi sains modern, posisi akhir atau *last author* sering kali ditempati oleh *Senior Author* atau *Principal Investigator* yang bertindak sebagai pembimbing utama atau penyedia dana riset (Hagen, 2010). Oleh karena itu, modifikasi model SDC diperlukan agar kredit kepakaran tidak meluruh terlalu rendah pada posisi penulis senior di akhir daftar.

2.2.6 Arsitektur Pencarian Berjenjang Dua Tahap

Desain sistem temu kembali informasi skala besar selalu dihadapkan pada dilema kompromi antara efektivitas akurasi dan efisiensi waktu eksekusi (Wang, Lin dan Metzler, 2011, hlm.105). Model pencarian tunggal yang kompleks sering kali membutuhkan biaya komputasi yang tinggi apabila dipaksakan untuk memindai seluruh dokumen dalam basis data. Guna mengatasi kendala tersebut, arsitektur pencarian berjenjang dua tahap atau *two-stage cascade retrieval* diterapkan untuk menyaring dokumen kandidat secara bertahap (Wang dkk., 2011, hlm.105).

Tahap pertama dalam arsitektur berjenjang ini berfokus pada efisiensi tinggi melalui penggunaan fungsi pencarian yang ringan digunakan untuk memangkas jutaan dokumen mentah menjadi daftar pendek berisi ratusan kandidat potensial (Nogueira dan Cho, 2020, hlm.1). Tahap kedua kemudian menerapkan fungsi pencarian yang jauh lebih kompleks dan kaya akan fitur untuk mengurutkan ulang daftar pendek tersebut demi menghasilkan akurasi pemeringkatan akhir yang presisi (Wang dkk., 2011, hlm.106).

Tantangan komputasi ini meningkat pada model pemrosesan bahasa alami modern berbasis *transformer* seperti BERT. Penggunaan BERT sebagai *cross-encoder*, yaitu metode di mana kueri dan dokumen digabungkan sejak awal untuk diumpamakan bersama ke dalam jaringan, menghasilkan overhead komputasi yang tinggi yang tidak praktis digunakan untuk memindai basis data berskala besar secara langsung (Reimers dan Gurevych, 2019, hlm.3982).

Sebagai solusinya, pendekatan *bi-encoder* melalui Sentence-BERT atau SBERT memetakan kueri dan dokumen secara independen ke dalam ruang vektor berdimensi tetap (Reimers dan Gurevych, 2019, hlm.3982). Hal ini memungkinkan tahap pertama pencarian dilakukan dengan menghitung jarak kedekatan kosinus secara instan pada basis data vektor yang telah diindeks. Tahap kedua kemudian bertugas melakukan kalkulasi metrik kemiripan yang lebih presisi hanya pada dokumen hasil saringan tahap pertama (Reimers dan Gurevych, 2019, hlm.3983).

2.2.7 Metode Evaluasi Berbasis Model Bahasa Besar sebagai Juri

Evaluasi berbasis preferensi manusia merupakan standar tertinggi dalam mengukur kualitas hasil pencarian yang bersifat subjektif dan dinamis. Namun, proses pengumpulan data preferensi langsung dari penilai manusia membutuhkan biaya operasional yang besar serta memakan waktu pengerjaan yang lama (Zheng dkk., 2023, hlm.2). Kendala skalabilitas tersebut mendorong urgensi penggunaan model bahasa besar sebagai juri otomatis guna mendekati preferensi manusia

(Zheng dkk., 2023, hlm.2). Metode evaluasi ini dikenal luas dengan istilah *LLM-as-a-Judge*.

Penggunaan model bahasa besar sebagai juri dinilai efektif karena model tersebut telah melalui proses penyelarasan instruksi yang intensif. Model bahasa besar tingkat lanjut terbukti mampu menghasilkan keputusan evaluasi yang konsisten dan selaras dengan penilaian pakar manusia (Zheng dkk., 2023, hlm.2).

Pengujian empiris menunjukkan bahwa tingkat kesepakatan keputusan antara juri otomatis dan pakar manusia mampu melampaui 80 persen (Zheng dkk., 2023, hlm.1). Persentase tersebut berada pada tingkat kesepakatan yang setara dengan konsistensi penilaian antar-manusia sendiri, sehingga menegaskan bahwa metode *LLM-as-a-Judge* merupakan alternatif evaluasi yang valid, efisien, dan memiliki skalabilitas tinggi (Zheng dkk., 2023, hlm.1).

2.2.8 Generasi Kueri Sintetis

Generasi kueri secara sintetis dari dokumen sumber merupakan metode yang telah mapan dan diakui secara luas dalam bidang temu kembali informasi. Pendekatan ini berakar pada metode ekspansi dokumen klasik seperti model *Doc2Query*, yang melatih model sekuensial untuk memprediksi kueri-kueri potensial yang relevan berdasarkan teks dari suatu dokumen sumber (Gospodinov, MacAvaney dan Macdonald, 2023, hlm.2). Melalui metode ini, representasi teks dari dokumen diperluas dengan menambahkan kueri buatan sebelum proses indeks dilakukan guna meningkatkan efektivitas temu kembali.

Perkembangan teknologi model bahasa besar memperluas kegunaan metode ini melalui pendekatan generasi kueri otomatis atau *synthetic query generation* (Chaudhary, Raman dan Bendersky, 2024, hlm.1645). Melalui pendekatan tersebut, model bahasa besar dipicu menggunakan instruksi khusus untuk menganalisis konteks dokumen masukan, seperti teks abstrak publikasi ilmiah, kemudian menghasilkan kueri spesifik yang relevan dengan dokumen tersebut (Chaudhary dkk., 2024, hlm.1645).

Guna menjamin kualitas kueri buatan yang dihasilkan, sistem pengujian dapat menerapkan fase penyaringan untuk mengeliminasi kueri dengan tingkat relevansi rendah terhadap dokumen aslinya (Gospodinov dkk., 2023, hlm.3). Metode generasi kueri berbasis dokumen ini terbukti menjadi solusi untuk mengevaluasi kinerja sistem temu kembali pada domain spesifik yang belum memiliki data latih maupun data uji standar (Chaudhary dkk., 2024, hlm.1647).

2.2.9 Sistem Penilaian ELO

Sistem penilaian ELO merupakan metode statistik yang pada awalnya dikembangkan oleh Arpad Elo untuk menentukan tingkat keterampilan relatif antar pemain dalam permainan dua pemain, paling terkenal adalah catur (Elo, 1966). Meskipun akar sejarahnya berada di bidang permainan, prinsip-prinsip ELO telah melampaui batas-batas awalnya dan kini diterapkan secara luas dalam berbagai konteks pemeringkatan yang didasarkan pada perbandingan berpasangan (*pairwise comparison*). Penerapan ini mencakup beragam domain

seperti olahraga, permainan video, dan, yang relevan untuk konteks penelitian ini, dalam evaluasi sistem berbasis preferensi pengguna.

Dalam sistem ELO setiap entitas yang dinilai memiliki peringkat numerik baru setelah setiap pertandingan. Perubahan dalam peringkat ditentukan oleh hasil langsung (menang, kalah, atau seri) dan peringkat relatif dari entitas yang dibandingkan. Kemenangan yang diraih melawan lawan yang memiliki peringkat jauh lebih tinggi akan menghasilkan peningkatan poin yang substansial. Kemenangan melawan lawan yang berperingkat lebih rendah memberikan sedikit tambahan poin.

Perhitungan dalam sistem ELO didasarkan pada dua persamaan utama. Pertama, persamaan untuk menghitung peluang menang (*Expected Score*) memprediksi probabilitas kemenangan suatu entitas berdasarkan perbedaan peringkat antara dua entitas yang bersaing. Jika peringkat entitas A adalah R_A dan peringkat entitas B adalah R_B maka peluang menang A (E_A) dirumuskan sebagai:

$$E_A = \frac{1}{1+10^{(R_B-R_A)/400}} \quad (2.2)$$

Nilai E_A merepresentasikan probabilitas kemenangan entitas A dan selalu berada dalam rentang 0 hingga 1. Apabila peringkat R_A sama dengan R_B , maka E_A akan bernilai 0.5. Kedua, setelah hasil perbandingan diketahui, peringkat entitas A akan diperbarui menggunakan persamaan peringkat baru (*New Rating*) yaitu:

$$R'_A = R_A + K (S_A - E_A) \quad (2.3)$$

R'_A adalah peringkat baru entitas A, sementara K konstanta yang memoderasi tingkat volatilitas peringkat. S_A adalah skor aktual yang diperoleh entitas A dari pertandingan.

Pendekatan ini telah terbukti efektif dalam mengevaluasi dan memeringkatkan model bahasa besar (LLM), seperti yang diimplementasikan pada platform *Chatbot Arena*. Platform tersebut menggunakan metodologi berbasis Elo untuk memeringkatkan berbagai LLM berdasarkan ribuan perbandingan anonim terhadap respons dari dua model oleh pengguna (Chiang et al., 2024).

BAB 3 METODOLOGI

3.1 Tipe Penelitian

Penelitian ini menggunakan tipe penelitian eksperimental kuantitatif dan analitis. Pendekatan ini digunakan untuk menguji hipotesis secara empiris, mengukur parameter performa secara matematis, serta menganalisis hubungan kausalitas dan kompromi (*trade-off*) antarvariabel komputasi. Penelitian ini menempatkan sistem pencarian yang dikonfigurasi sebagai instrumen pengujian (*testbed*) terkontrol untuk mengumpulkan data performa komputasi dan kualitas relevansi semantik.

Dampak manipulasi variabel bebas terhadap variabel terikat dianalisis berdasarkan beberapa definisi operasional. Variabel dalam penelitian ini dibagi menjadi dua kategori, yaitu (1) variabel bebas, yang berupa panjang dimensi representasi vektor Matryoshka dengan variasi nilai sebesar 64, 128, 256, 512, dan 768; serta (2) variabel terikat, yang mencakup efisiensi waktu eksekusi atau latensi yang diukur melalui durasi komputasi pencarian vektor dalam satuan detik, dan kualitas relevansi semantik atau skor Elo yang diukur melalui skor preferensi relatif lalu dikonversi menjadi metrik pemeringkatan Elo berdasarkan evaluasi juri otomatis.

3.2 Pengumpulan Data

Proses pengumpulan data sekunder dilakukan secara terprogram memanfaatkan fungsionalitas ekspor data terstruktur dari portal resmi Scopus. Luaran hasil unduhan berupa berkas dengan format *Comma-Separated Values* (CSV) yang menyimpan atribut metadata publikasi. Atribut yang diekstraksi meliputi judul artikel ilmiah (*title*), abstrak (*abstract*), daftar nama penulis (*authors*), ID unik penulis (*Scopus ID*), tahun publikasi, indeks sitasi, pengenalan objek digital (*digital object identifier/DOI*), dan afiliasi institusi peneliti.

Sementara itu, untuk menguji sistem pencarian, dirancang sebuah prosedur pembuatan 100 kueri uji secara otomatis berdasarkan dokumen yang telah tersedia. Prosedur ini diawali dengan menyaring 100 artikel ilmiah berindeks sitasi tertinggi dari basis data. Selanjutnya, teks abstrak dari masing-masing artikel tersebut dikirimkan ke LLM Gemma melalui perintah instruksi terstruktur untuk merumuskan satu kueri pencarian alami yang berorientasi pada solusi praktis. Pendekatan pencarian item yang diketahui ini diterapkan untuk memastikan bahwa minimal terdapat satu dokumen relevan dan satu pakar nyata di dalam basis data untuk setiap kueri uji yang diajukan.

3.3 Metode *Preprocessing*

Metode *preprocessing* dirancang untuk mentransformasikan data teks mentah menjadi representasi vektor numerik melalui langkah-langkah berikut:

1. *Data Cleaning* dilakukan pada teks abstrak mentah secara terprogram menggunakan ekspresi reguler (*regular expression*/regex) untuk menghapus deret teks hak cipta (*copyright headers*), seperti teks "© 2024 Elsevier Ltd. All rights reserved.", yang terdapat pada bagian akhir abstrak publikasi Scopus agar representasi vektor semantik murni mencerminkan konten ilmiah tanpa gangguan teks hukum.
2. Judul dan abstrak artikel digabungkan menjadi satu dokumen teks menggunakan karakter spasi tunggal.
3. Pembuatan vektor diproses oleh layanan *text-embeddings-inference* (TEI) menggunakan model *nomic-embed-text-v1.5* untuk memproduksi *dense vector* berdimensi penuh (768 dimensi).
4. Penyimpanan data dilakukan dengan memasukkan vektor hasil komputasi secara langsung ke dalam tabel *articles* pada pangkalan data PostgreSQL menggunakan tipe data *VECTOR(768)* yang terindeks dan terhubung ke tabel *authors* melalui tabel *articles_authors*.

3.4 Metode Evaluasi

Evaluasi kualitas pencarian pakar dilakukan menggunakan metode *blind pairwise comparison* dengan pendekatan *LLM-as-a-Judge*. Pendekatan ini dipilih untuk mengatasi keterbatasan biaya serta menjaga konsistensi penilaian yang sering kali subjektif jika dilakukan oleh manusia. Eksperimen dijalankan secara otomatis melalui sebuah program pengujian dengan urutan langkah sebagai berikut:

1. Tahap pertama dimulai dengan memproses 100 kueri uji sintetis yang mewakili domain permasalahan riset ke dalam mesin pencari.
2. Selanjutnya, pencarian dijalankan menggunakan kelima variasi dimensi representasi dari model *Matryoshka embeddings*, yaitu 64, 128, 256, 512, dan 768 dimensi.
3. Hasil pencarian dari setiap pasangan dimensi kemudian dipasangkan secara kombinatorik dan dianonimkan menjadi Pilihan A dan Pilihan B guna menghindari bias urutan dimensi saat dinilai.
4. Setelah itu, pasangan hasil pencarian beserta deskripsi kueri dikirimkan ke LLM eksternal yang bertindak sebagai juri otomatis untuk menentukan keputusan menang, kalah, atau seri berdasarkan parameter relevansi semantik dan bobot sitasi.
5. Tahap terakhir adalah menyimpan seluruh keputusan juri beserta catatan waktu eksekusi pencarian ke dalam basis data untuk analisis statistik.

3.5 Metode Analisis Data

Data hasil evaluasi yang telah terkumpul dianalisis secara mendalam menggunakan beberapa teknik statistik inferensial dan optimasi multiobjektif. Analisis pertama berfokus pada efisiensi waktu eksekusi sistem. Pengujian ini menggunakan metode *One-way Analysis of Variance* (ANOVA) untuk mendeteksi adanya perbedaan rata-rata latensi yang signifikan secara statistik di antara berbagai tingkat dimensi. Apabila ditemukan perbedaan yang signifikan, analisis dilanjutkan dengan uji *Tukey's Honestly Significant Difference* (HSD) sebagai langkah *post hoc* untuk mengidentifikasi secara spesifik pasangan dimensi mana saja yang memiliki perbedaan kecepatan eksekusi yang nyata.

Analisis kedua berfokus pada kualitas hasil pencarian yang diukur melalui konversi keputusan juri menjadi skor Elo. Untuk mendapatkan estimasi skor yang stabil dan representatif, digunakan simulasi *bootstrap* Monte Carlo sebanyak 1.000 iterasi. Proses ini menghasilkan nilai rata-rata skor Elo beserta rentang interval kepercayaan 95 persen dari masing-masing dimensi. Hubungan antara peningkatan ukuran dimensi terhadap kualitas hasil pencarian kemudian diukur signifikansinya menggunakan koefisien korelasi peringkat Spearman. Sebagai langkah akhir, dilakukan pemetaan batas efisiensi Pareto (*Pareto Frontier*). Pemetaan ini bertujuan untuk mengidentifikasi konfigurasi dimensi optimal yang mampu memberikan kinerja pencarian terbaik dengan biaya komputasi yang paling efisien.

BAB 4 PERANCANGAN

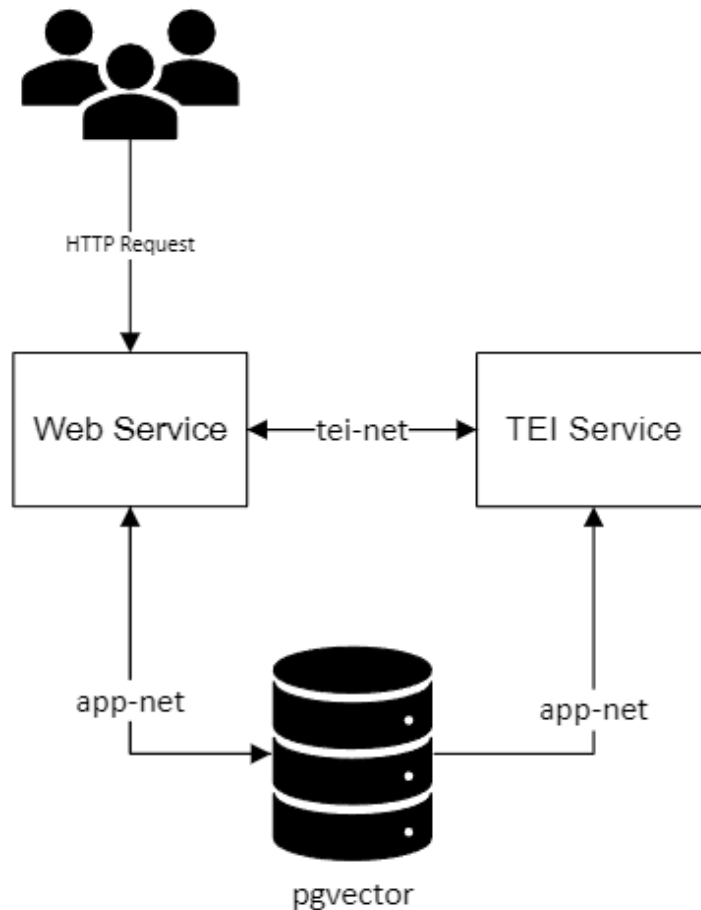
4.1 Deskripsi Umum

Sistem yang dirancang dalam penelitian ini adalah sebuah prototipe aplikasi pencarian pakar berbasis *Document-based Model* yang bertujuan untuk menganalisis pengaruh variasi panjang dimensi embedding Matryoshka terhadap kualitas hasil dan waktu eksekusi. Kueri berupa teks dari pengguna diterima oleh sistem, kemudian diproses untuk menghasilkan dua daftar peringkat pakar yang dihasilkan dari dua konfigurasi dimensi embedding yang berbeda secara acak. Hasil tersebut disajikan dalam format perbandingan berdampingan (*pairwise comparison*) untuk dievaluasi oleh pengguna, guna mengukur preferensi dan kualitas hasil secara subjektif.

Data sumber untuk model kepakaran dalam penelitian ini diperoleh dari basis data publikasi ilmiah Scopus. Proses pengumpulan data diawali dengan mengakses portal Scopus <https://www.scopus.com/pages/home>, melakukan pencarian organisasi dengan kata kunci "Indonesia", dan mengurutkan hasilnya berdasarkan jumlah publikasi terbanyak. Dari daftar tersebut, dipilih 10 institusi teratas, yaitu: Badan Riset dan Inovasi Nasional (BRIN), Institut Pertanian Bogor (IPB), Institut Teknologi Bandung (ITB), Institut Teknologi Sepuluh Nopember (ITS), Universitas Brawijaya (UB), Universitas Gadjah Mada (UGM), Universitas Indonesia (UI), Universitas Airlangga (Unair), Universitas Diponegoro (Undip), dan Universitas Padjadjaran (Unpad). Untuk masing-masing institusi, dilakukan ekspor metadata artikel ilmiah dalam format CSV untuk rentang waktu publikasi 2020 hingga 2025, yang kemudian menjadi korpus utama untuk membangun model kepakaran sistem.

4.2 Arsitektur Sistem

Arsitektur sistem ini mengadopsi model *micro-service* yang mengintegrasikan layanan backend dengan basis data vektor dan layanan inferensi eksternal.



Gambar 4.1 Arsitektur Sistem Pencarian Pakar

4.2.1 Komponen Utama Sistem

Arsitektur sistem ini terdiri dari tiga komponen utama yang saling berinteraksi:

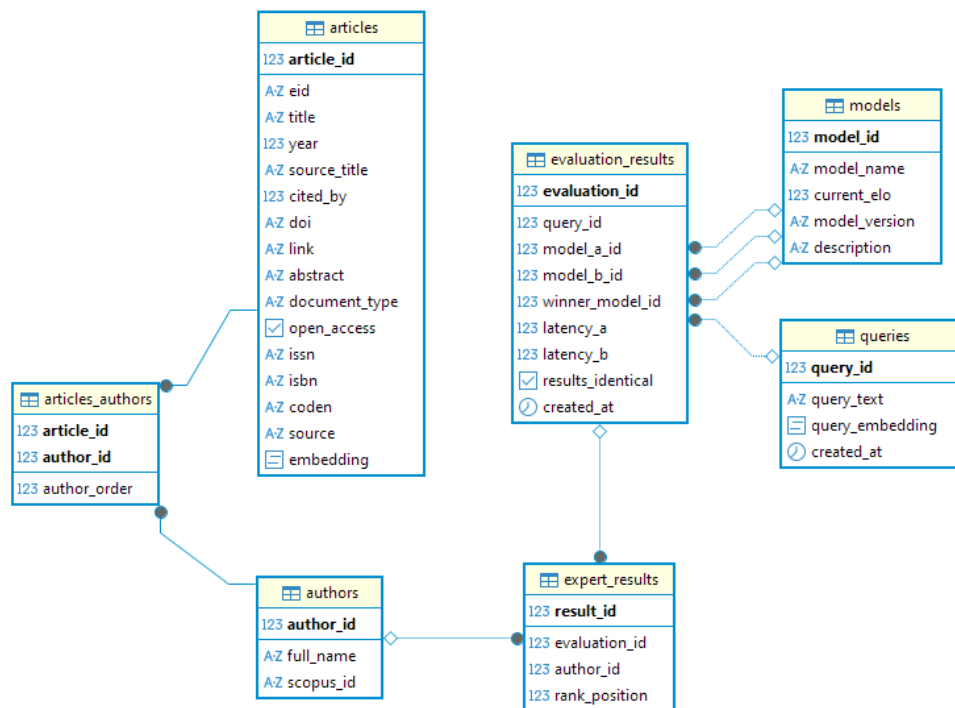
1. Web Service. Dibangun menggunakan framework Flask, berfungsi sebagai orkestrator utama yang menangani logika bisnis, permintaan API, dan alur kerja pencarian.
2. Basis Data. Menggunakan PostgreSQL versi 17 dengan ekstensi pgvector untuk penyimpanan dan kueri efisien terhadap dense vector embedding.
3. Text Embedding Inference (TEI) Service. Layanan dari Hugging Face (versi 1.7.4) yang berfungsi mengubah data teks mentah menjadi representasi vektor semantik menggunakan model `nomi-`

ai/nomic-embed-text-v1.5 yang telah dilatih dengan metode *Matryoshka Representation Learning*.

4.2.2 Skema Basis Data

Skema basis data dirancang secara relasional untuk mendukung fungsi pencarian pakar dan evaluasi sistem. Berikut adalah rincian setiap tabel beserta fungsinya:

1. `articles`. Tabel inti yang menyimpan metadata setiap artikel ilmiah, seperti judul, abstrak, tahun, dan DOI. Tabel ini memiliki kolom embedding dengan tipe data `VECTOR(768)` untuk menyimpan representasi vektor dari setiap artikel.
2. `authors`. Tabel ini menyimpan informasi unik untuk setiap penulis, diidentifikasi oleh `scopus_id`, untuk menghindari redundansi data dan memudahkan agregasi.
3. `articles_authors`. Tabel penghubung (junction table) yang memetakan hubungan many-to-many antara artikel dan penulis. Tabel ini juga menyimpan kolom `author_order` yang berfungsi untuk memberikan bobot pada kontribusi penulis dalam proses penilaian kepakaran.
4. `queries`. Mencatat setiap teks kueri yang dimasukkan oleh pengguna beserta embedding-nya. Tabel ini berfungsi untuk melacak histori pencarian dan menghubungkan hasil evaluasi dengan kueri aslinya.
5. `models`. Tabel konfigurasi yang mendefinisikan berbagai model (variasi dimensi MRL) yang akan dievaluasi, seperti `64_dim`, `128_dim`, hingga `768_dim`. Tabel ini juga menyimpan skor ELO terkini untuk setiap model.
6. `evaluation_results`. Tabel ini berfungsi mencatat hasil dari setiap perbandingan pairwise, termasuk ID kueri, dua model yang dibandingkan (`model_a_id`, `model_b_id`), model pemenang, dan latensi masing-masing.
7. `expert_results`. Tabel ini berfungsi menyimpan daftar peringkat pakar yang dihasilkan untuk setiap evaluasi, menghubungkannya ke `evaluation_id` untuk analisis lebih lanjut.



Gambar 4.2 Relational Schema Basis Data Sistem Pencarian Pakar

4.2.3 Interaksi Antar Komponen

Interaksi antar komponen difasilitasi oleh jaringan internal Docker. Seluruh layanan (Web, DB, TEI) dijalankan dalam jaringan yang sama yang memungkinkan komunikasi melalui nama layanan sebagai *hostname*. Alur interaksi untuk satu permintaan pencarian adalah sebagai berikut:

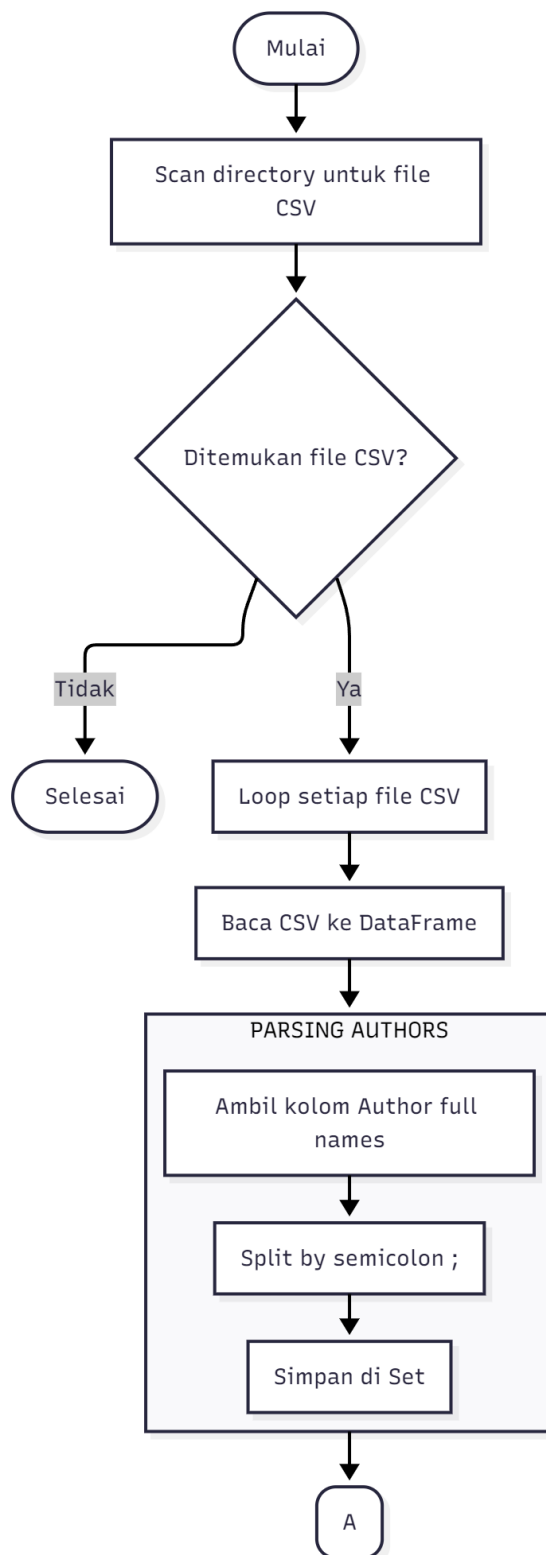
1. Teks kueri dikirimkan oleh *Web Service* ke *TEI service* melalui jaringan Docker `http://tei-embedding/embed` untuk mendapatkan vektor embedding kueri.
2. Koneksi dilakukan ke basis data *db* untuk menjalankan dua kueri SQL. Setiap kueri melakukan pencarian *cosine similarity* antara vektor kueri dan vektor artikel menggunakan fungsi *sub_vector* dengan dimensi yang berbeda.
3. Logika *reranking* dijalankan dengan mengevaluasi kembali kemiripan vektor pada dimensi penuh, melakukan agregasi skor untuk setiap pakar, dan menyusun respons akhir dalam format JSON untuk dikirimkan ke antarmuka pengguna.

4.3 Alur Kerja Sistem

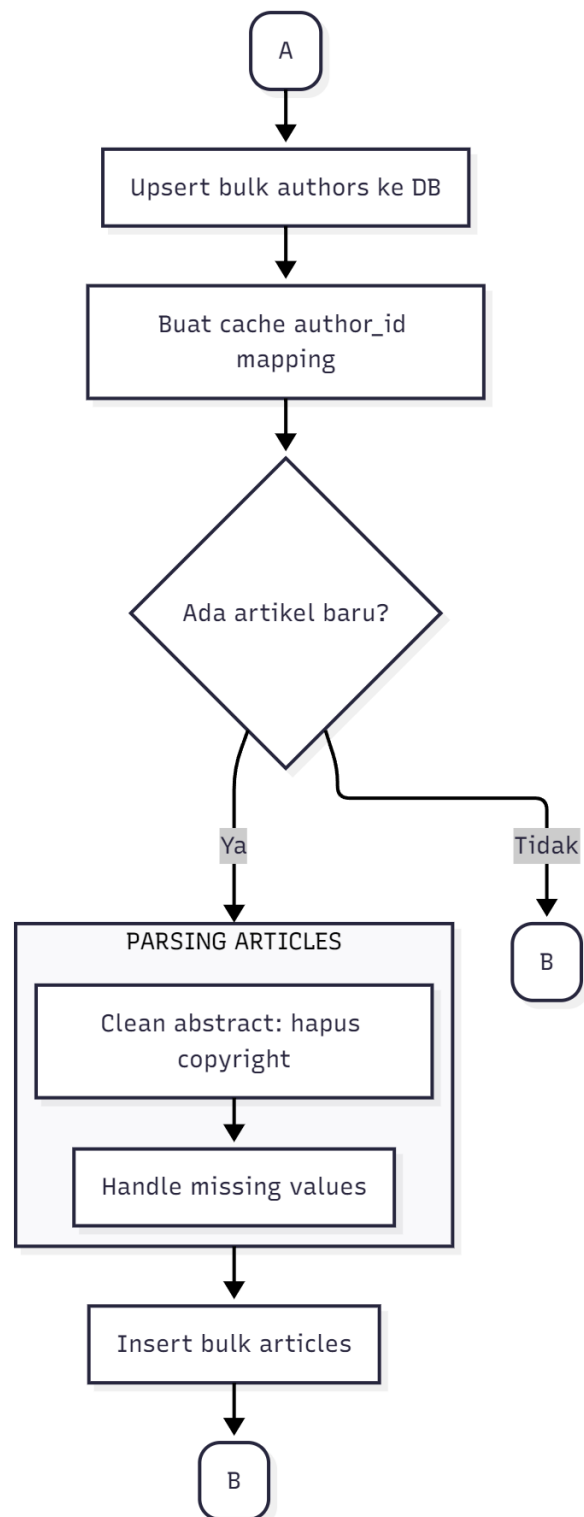
Alur kerja sistem dibagi menjadi tahapan *Data Ingestion*, Vektorisasi, dan Pencarian Pakar yang terstruktur.

4.3.1 Data Ingestion

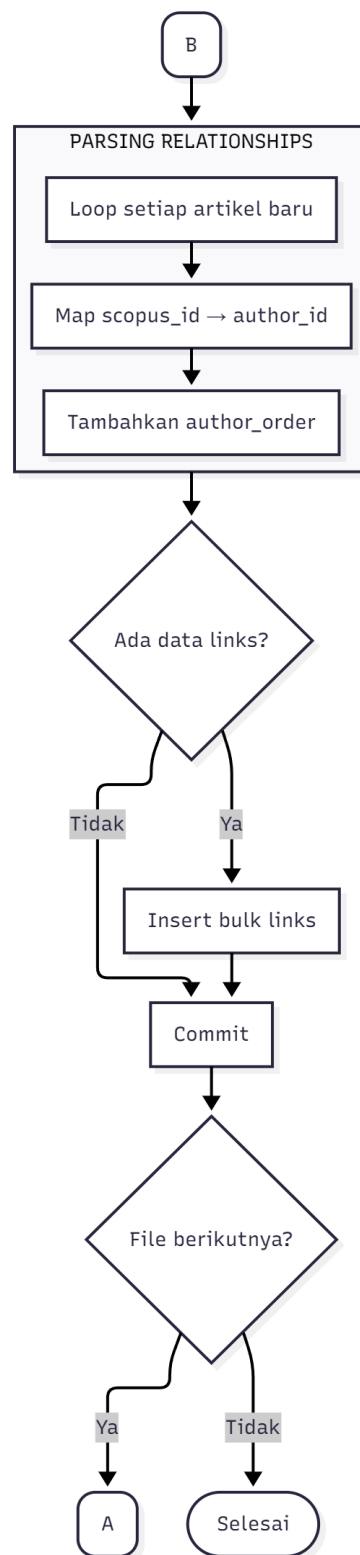
Data Ingestion adalah proses yang memuat data Scopus mentah ke dalam basis data dan menyiapkan struktur data relasional. Proses ini dimulai dengan pembacaan semua file CSV data publikasi. Kemudian, sistem mengekstrak semua entitas penulis unik dan memastikan entitas tersebut dicatat dalam tabel penulis. Setelah itu, sistem mengidentifikasi artikel-artikel baru, melakukan pembersihan dasar pada teks abstrak dengan menghilangkan informasi hak cipta, dan memasukkannya ke dalam tabel artikel. Terakhir, hubungan antara setiap artikel dan penulisnya dicatat, termasuk urutan penulis yang akan digunakan dalam penilaian kepakaran.



Gambar 4.3 Diagram Alur *Data Ingestion*



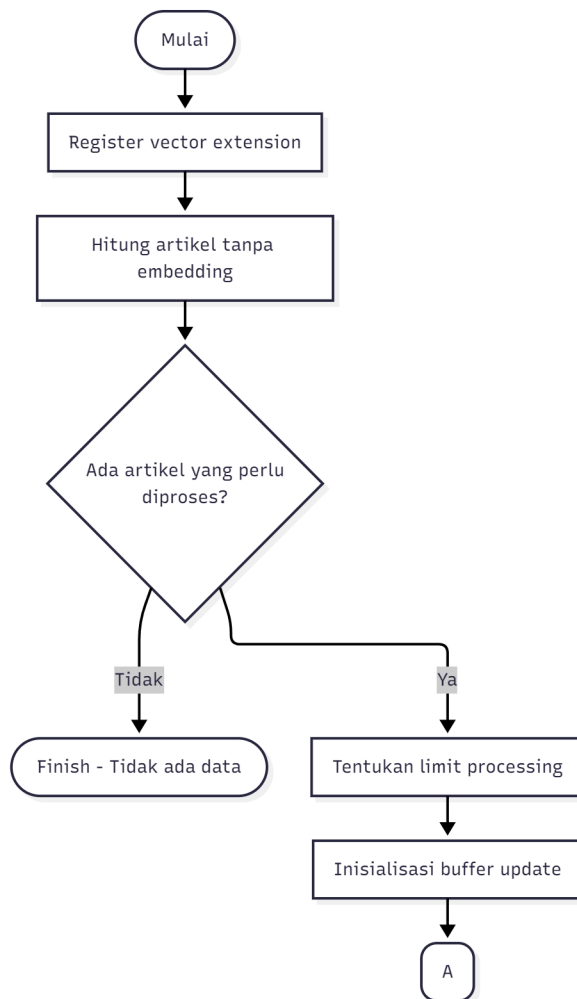
Gambar 4.4 Diagram Alur *Data Ingestion*



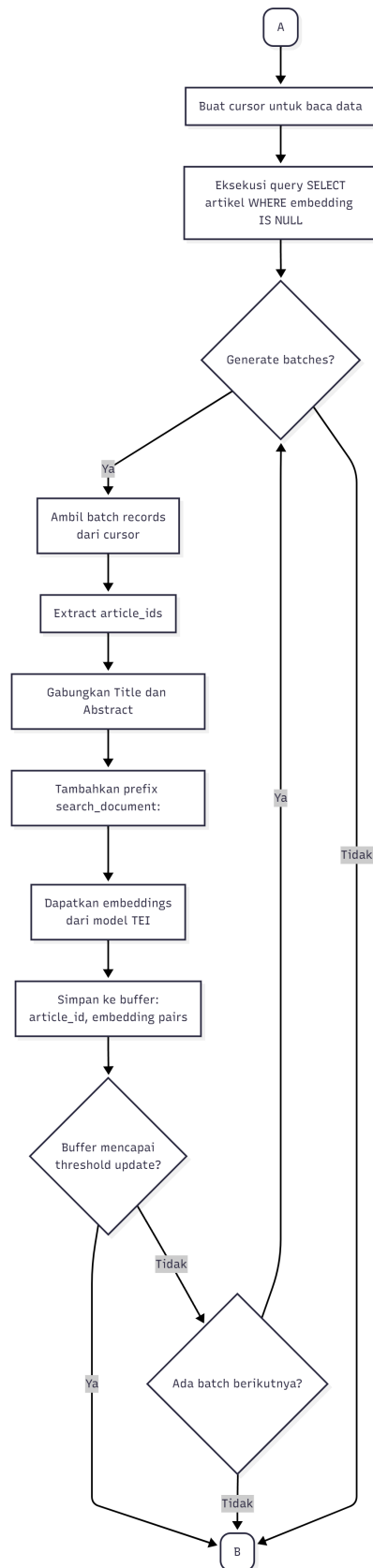
Gambar 4.5 Diagram Alur *Data Ingestion*

4.3.2 Alur Vektorisasi Dokumen

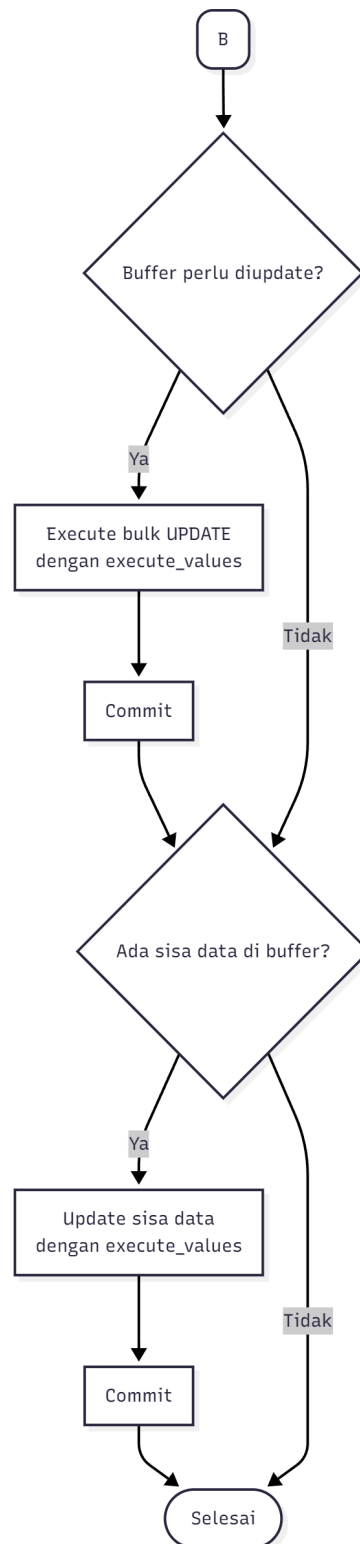
Alur vektorisasi merupakan proses yang mengubah data tekstual yang sudah di-ingest menjadi representasi vektor semantik. Sistem secara bertahap memilih dokumen yang belum memiliki embedding dari basis data. Dokumen-dokumen ini diproses dalam batch, melalui pengiriman teks secara kolektif ke *TEI service*. Layanan tersebut menghasilkan vektor embedding Matryoshka berdimensi 768. Setelah vektor diterima, sistem memperbarui kolom embedding pada tabel artikel terkait di basis data. Proses ini berlangsung hingga semua dokumen dalam korpus telah memiliki representasi vektor.



Gambar 4.6 Diagram Alur Vektorisasi Dokumen



Gambar 4.7 Diagram Alur Vektorisasi Dokumen



Gambar 4.8 Diagram Alur Vektorisasi Dokumen

4.3.3 Alur Pencarian Pakar

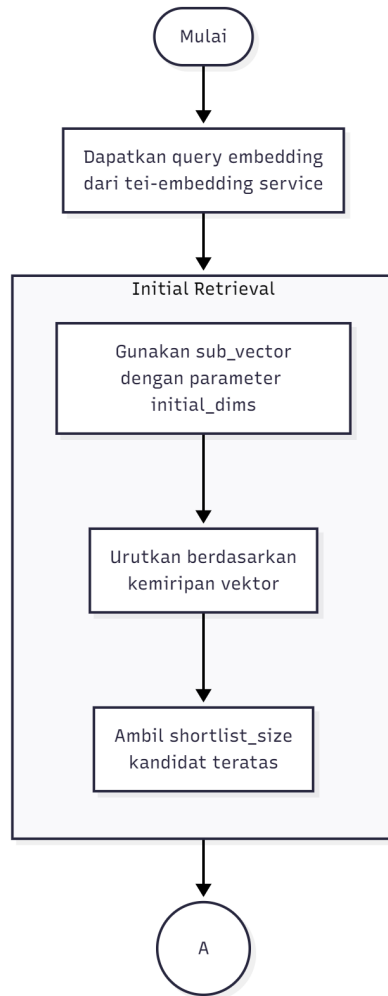
Proses penemuan pakar dalam penelitian ini dirancang dengan mengimplementasikan arsitektur pencarian berjenjang dua tahap guna menyeimbangkan performa waktu respon dan kualitas hasil pemeringkatan. Alur kerja pencarian pakar ini dibagi menjadi dua fase utama, yaitu fase penyaringan cepat atau *shortlisting* sebagai tahap pertama, dan fase pengurutan ulang atau *reranking* sebagai tahap kedua.

Tahap pertama bertujuan untuk melakukan pemangkasan dokumen secara luas terhadap seluruh artikel ilmiah yang tersimpan di dalam basis data PostgreSQL. Pada fase ini, sistem melakukan pencarian tetangga terdekat menggunakan fungsi kustom `sub_vector` pada indeks HNSW yang dibangun di atas dimensi *Matryoshka* berukuran rendah, yaitu 128 dimensi. Reduksi dimensi ini meminimalkan beban kalkulasi memori dan memangkas waktu pemindaian basis data. Output dari tahap pertama ini adalah daftar pendek berisi 100 dokumen kandidat yang memiliki jarak kosinus terdekat dengan vektor kueri.

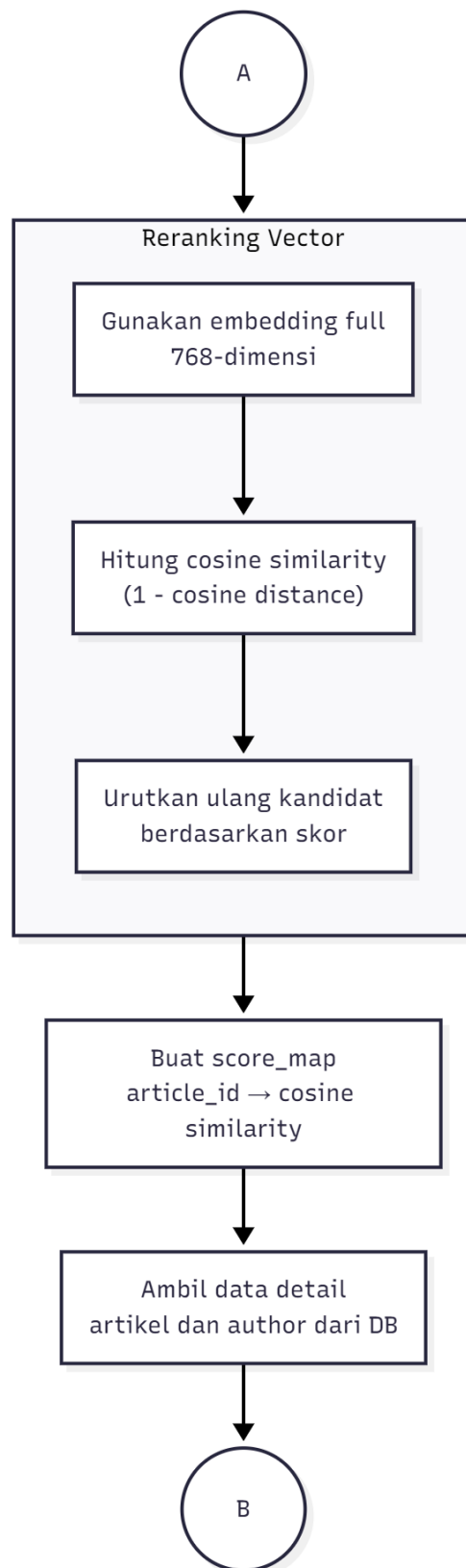
Pada tahap kedua daftar pendek berisi 100 dokumen diambil untuk dievaluasi kembali menggunakan representasi semantik dimensi penuh, yaitu 768 dimensi. Pada tahap ini, komputasi metrik kemiripan kosinus murni dilakukan secara presisi guna menghasilkan nilai kedekatan semantik yang akurat.

Dengan membatasi kalkulasi dimensi penuh hanya pada 100 dokumen kandidat hasil saringan tahap pertama, bottleneck komputasi dimensi tinggi berhasil dieliminasi tanpa mengorbankan kelengkapan fitur semantik yang dibutuhkan untuk pemeringkatan akhir.

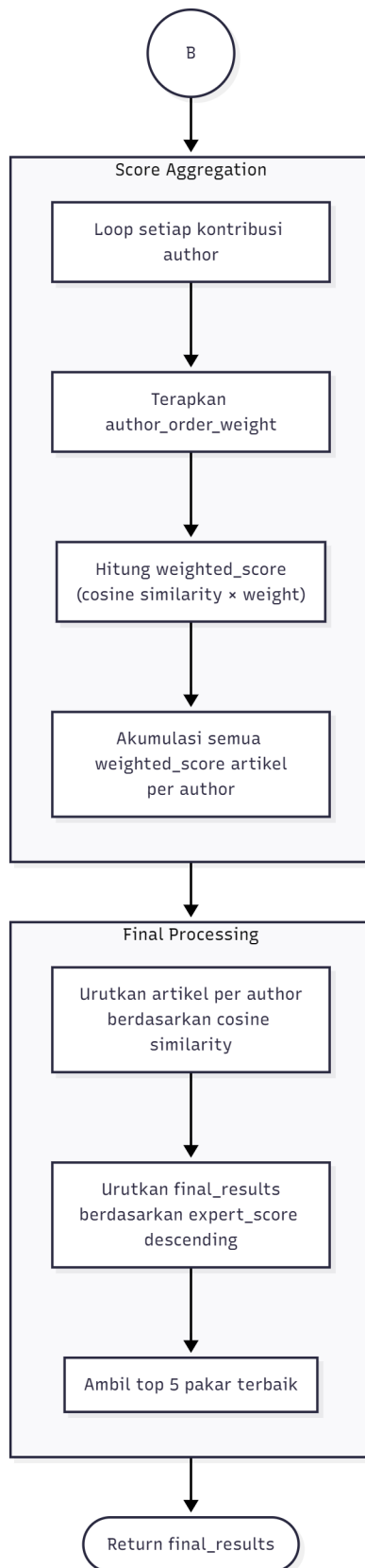
Skor hasil pengurutan ulang dari setiap dokumen tersebut kemudian diagregasikan ke masing-masing penulis menggunakan skema pembobotan posisi kepengarangan. Proses agregasi ini menghasilkan daftar final peringkat kepakaran peneliti yang akan dikirimkan kembali ke antarmuka pengguna dalam format JSON. Visualisasi diagram alur dari proses dua tahap ini digambarkan secara terperinci pada Gambar 4.9.



Gambar 4.9 Diagram Alur Pencarian Pakar



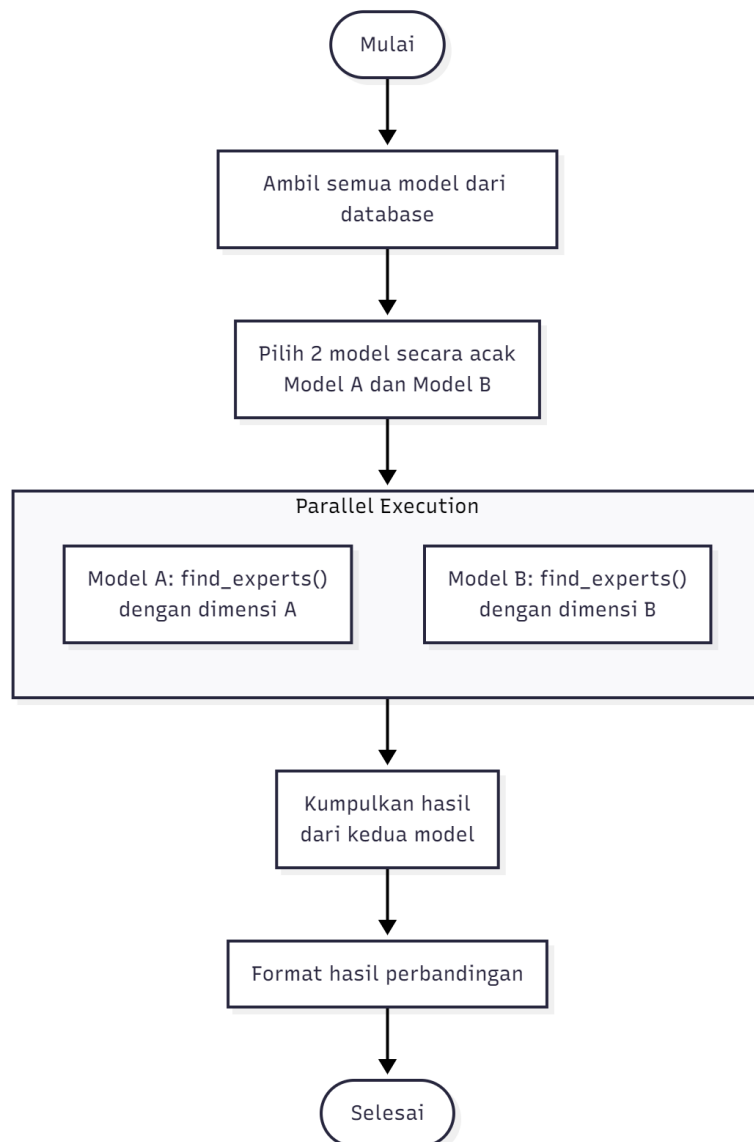
Gambar 4.10 Diagram Alur Pencarian Pakar



Gambar 4.11 Diagram Alur Pencarian Pakar

4.3.4 Alur Evaluasi *Pairwise*

Alur evaluasi pairwise adalah mekanisme utama untuk mengukur kualitas hasil. Ketika pengguna mengajukan kueri, sistem secara acak memilih dua konfigurasi dimensi MRL yang berbeda untuk dibandingkan (Model A dan Model B). Kedua konfigurasi ini menjalankan Alur Pencarian Pakar (4.3.3) secara bersamaan untuk kueri yang sama. Hasil peringkat pakar dari Model A dan Model B disajikan berdampingan secara anonim di antarmuka web. Pilihan preferensi pengguna (A lebih baik, B lebih baik, atau Seri) kemudian dicatat, bersama dengan data pendukungnya (dimensi, waktu eksekusi, dan daftar pakar), ke dalam tabel log evaluasi.



Gambar 4.12 Alur Evaluasi Pairwise

4.4 Expertise Modeling

Pemodelan kepakaran dalam sistem ini mengadopsi pendekatan *Document-based Model*, yaitu metode penilaian tingkat keahlian peneliti berdasarkan relevansi semantik karya ilmiah yang dipublikasikannya. Skor kepakaran seorang peneliti, yang dinyatakan sebagai *ExpertScore*, dihitung dengan mengakumulasi skor relevansi dari setiap artikel ilmiah terkait yang ia tulis.

Untuk memitigasi bias ekualisasi akibat pemberian kredit yang setara kepada penulis pendamping dan penulis utama, diterapkan skema pembobotan *Linear SDC* dengan Batas Minimum. Skema pembobotan ini dirancang untuk menjembatani kesenjangan antara kontribusi teknis penulis utama dan kontribusi strategis penulis senior pada urutan akhir. Formula pembobotan urutan penulis (w_i) didefinisikan sebagai berikut:

$$w_i = \max(1 - 0.2(i - 1), 0.4) \quad (4.1)$$

Simbol i pada persamaan di atas mewakili posisi urutan penulis pada artikel terkait. Formula tersebut menghasilkan peluruhan linier sebesar 0.2 bagi penulis utama hingga penulis ketiga, dengan rincian bobot berturut-turut sebesar 1,0 untuk penulis pertama, 0,8 untuk penulis kedua, dan 0,6 untuk penulis ketiga.

Namun, bobot dikunci pada nilai minimum 0,4 untuk penulis keempat dan seterusnya. Batas minimum ini berfungsi sebagai *Seniority Floor* untuk mengapresiasi kontribusi *Principal Investigator* atau pembimbing riset yang umumnya berada di posisi kepengarangan akhir (Hagen, 2010)

Berdasarkan pembobotan tersebut, skor akhir kepakaran peneliti p dirumuskan sebagai berikut:

$$ExpertScore_p = \sum_{i \in D_p} Score_p \times w_i \quad (4.2)$$

Simbol D_p melambangkan himpunan dokumen relevan yang ditulis oleh peneliti p , sedangkan $Score_i$ melambangkan skor kemiripan semantik dokumen i pasca-proses *reranking*.

4.5 Perhitungan Manual

Alur kerja pencarian pakar dari input kueri hingga peringkat akhir diilustrasikan menggunakan dimensi *shortlisting* $m_{initial} = 128$, dimensi penuh $D = 768$ untuk *reranking* dan jumlah dokumen kandidat yang diteruskan ke tahap *Reranking* $K = 100$

4.5.1 Persiapan Input

Sebelum kueri diproses oleh model embedding, sistem menerapkan *prompt engineering* berupa prefix (`search_query:`) untuk mengarahkan model menghasilkan representasi vektor yang optimal untuk tugas pencarian.

Tabel 4.1 Contoh Representasi Vektor Kueri Setelah Penambahan Prefix

Teks Kueri Mentah (Q)	Kueri Ber-Prefix	Vektor Kueri Penuh (q)
"Impact of microplastics on marine food chains"	"search_query: Impact of microplastics on marine food chains"	$[q_1, q_2, \dots, q_{768}]$

Vektor dokumen (d) sudah tersimpan dalam basis data dengan dimensi penuh D .

Tabel 4.2 Contoh Representasi Vektor Dokumen dari Basis Data

ID Artikel (α)	Judul Dokumen	Abstrak Dokumen	Vektor Dokumen Penuh (d)
9021	Microplastic accumulation in respiratory...	Microplastics pose a significant environmental..	$[d_1, d_2, \dots, d_{768}]$
1005	An empirical analysis of social media usage,...	Youth participation in political...	$[d_1, d_2, \dots, d_{768}]$
25302	Physiological and psychological effects..	The benefits of park therapy have been...	$[d_1, d_2, \dots, d_{768}]$
...	...	...	...

Vektor penuh q dan d dipotong ke dimensi $m_{initial}$. Setelah pemotongan, sub-vektor tersebut menjalani proses normalisasi ulang. Misalkan v adalah sub-vektor yang dipotong. Sub-vektor yang dinormalisasi v^* dihitung sebagai:

$$v_i^* = \frac{v_i}{\sqrt{\sum_{j=1}^m v_j}} \quad (4.3)$$

v_i adalah komponen ke- i dari sub-vektor yang dipotong dan $\sqrt{\sum_{j=1}^m v_j}$ adalah L_2 norm dari sub-vektor yang dipotong. Vektor q dan d yang digunakan dalam *shortlisting* adalah yang dinormalisasi ulang.

Tabel 4.3 Contoh Vektor Terpotong dan Ternormalisasi

Vektor Asli	Vektor Terpotong dan Ternormalisasi
$q = q_{1:768}$	$q_{1:128}^* = [q_1^*, q_2^*, \dots, q_{128}^*]$
$d = d_{1:768}$	$d_{1:128}^* = [d_1^*, d_2^*, \dots, d_{128}^*]$

4.5.2 Shortlisting

Sistem melakukan pencarian tetangga terdekat dengan menghitung *dot product* antara sub-vektor kueri $q_{1:128}^*$ sub-vektor dokumen $d_{1:128}^*$.

$$Score_{initial,a} = d_{a,1:128}^* \cdot q_{1:128}^* \quad (4.4)$$

Asumsikan komponen awal dari $d_{9021,1:128}$ dan q setelah dipotong dan dinormalisasi adalah:

$$d_{9021,1:128}^* = [d_1^*, d_2^*, \dots, d_{128}^*] \quad (4.5)$$

$$q_{1:128}^* = [q_1^*, q_2^*, \dots, q_{128}^*] \quad (4.6)$$

Tabel 4.4 Hasil Shortlisting

Peringkat	ID Artikel (a)	$Score_{initial}$	ID Penulis Terkait
1	9021	0.887	301, 302, 303
2	1005	0.642	302, 305
...	...	...	...
100	...	...	...

4.5.3 Reranking

Dokumen dari shortlist diurutkan ulang menggunakan dimensi penuh D untuk mendapatkan skor relevansi. Skor relevansi dihitung dengan *dot product* antara vektor query q dan vektor dokumen d .

$$Score_a = d \cdot q \quad (4.7)$$

Tabel 4.5 Hasil Reranking

Peringkat	ID Artikel (a)	$Score_a$
1	9021	0.910
2	1005	0.531
...	...	...
100	...	...

Setelah *Reranking* skor relevansi dokumen menjadi lebih akurat karena vektor q dan vektor d memiliki informasi semantik yang lebih kaya.

4.5.4 Agregasi Skor Kepakaran

Skor kepakaran (*ExpertScore*) dihitung menggunakan pendekatan CombSUM (*Combination Sum*) yang dimodifikasi dengan mengaplikasikan *Author Order Weighting* pada skor relevansi setiap dokumen. Pembobotan ini menentukan besarnya kontribusi artikel terhadap kepakaran seorang penulis berdasarkan posisi mereka dalam daftar penulis.

Tabel 4.6 Skema Author Order Weighting

Urutan Penulis	Bobot (<i>AuthorOrderWeight</i>)
1	1.0
2	0.8
3	0.6
≥ 4	0.4

Setiap kontribusi artikel dihitung dengan:

$$WeightedScore_a = Score_a \cdot AuthorOrderWeight \quad (4.8)$$

Tabel 4.7 Perhitungan Weighted Score

ID Penulis	ID Artikel (a)	$Score_a$	Urutan	Bobot	$WeightedScore_a$
301	9021	0.910	1	1.0	0.910
302	9021	0.910	2	0.8	0.728
303	9021	0.910	3	0.6	0.546
302	1005	0.862	1	1.0	0.862
...	...	...	...	...	...

$ExpertScore$ adalah hasil agregasi dari semua $WeightedScore_a$ terkait penulis p .

$$ExpertScore_p = \sum WeightedScore_a \quad (4.9)$$

Tabel 4.8 Agregasi dan Pemeringkatan Akhir Skor Kepakaran

Peringkat	ID Penulis	Nama Pakar	Kalkulasi Agregasi	<i>ExpertScore</i>
1	301	Saud, M.	$0.910 + \dots$	2.34
2	302	Intyaswati, D.	$0.728 + \dots$	1.42
...	...	...	...	...

BAB 5 IMPLEMENTASI

5.1 Deskripsi Umum Implementasi Sistem

Bagian ini menjelaskan modul-modul utama yang diimplementasikan dalam sistem pencarian pakar berbasis *Document-based Model* menggunakan MRL. Implementasi sistem ini dibagi menjadi empat bagian utama:

1. Tahap pertama meliputi implementasi basis data relasional PostgreSQL yang dilengkapi dengan ekstensi pgvector untuk menyimpan representasi vektor artikel ilmiah, serta fungsi pemotongan vektor (*vector slicing*).
2. Tahap kedua berfokus pada modul pencarian pakar yang mengimplementasikan metode pencarian *two-stage retrieval* dan pemodelan kepakaran berbasis *Linear SDC*.
3. Tahap ketiga mencakup implementasi layanan web berupa antarmuka pengguna berbasis Flask dan modul logging untuk mencatat preferensi evaluasi.
4. Tahap keempat merupakan implementasi *automatic evaluation pipeline* yang dijalankan menggunakan generator kueri sintetis dan juri otomatis berbasis LLM.

5.2 Implementasi Basis Data dan *Pre-processing* Data

5.2.1 Implementasi *Ingestion* dan *Pre-processing* Data Scopus

Algoritma 1: <i>Ingestion</i> dan Pra-pemrosesan Data Scopus	
1	INPUT: Scopus CSV
2	FOR each row in Scopus CSV:
3	abstract = regex_remove(row.abstract, copyright_pattern)
4	FOR each author in row.authors with index:
5	id, name = regex_extract(author, pattern)
6	all_authors.add(id, name)
7	articles_authors.add(row.eid, id, author_order = index + 1)
8	END FOR
9	END FOR
10	bulk_upsert(all_authors) into "authors" table
11	bulk_insert(articles) into "articles" table
12	bulk_insert(articles_authors) into "articles_authors" table

Modul ini bertanggung jawab untuk membaca data mentah publikasi ilmiah dari berkas CSV hasil ekspor Scopus, melakukan pembersihan data dari teks hak cipta, mengekstrak data penulis, serta menyimpannya ke dalam struktur data relasional. Penjelasan tahapan program pada Algoritma 1 di atas adalah sebagai berikut:

1. Baris 1 menunjukkan input yang dibutuhkan oleh modul ini, yaitu berkas CSV mentah hasil ekspor dari portal Scopus.
2. Baris 2 memulai proses iterasi untuk membaca data artikel ilmiah baris demi baris dari berkas CSV.
3. Baris 3 menjalankan fungsi pembersihan teks abstrak menggunakan *regular expression* untuk menghapus teks informasi hak cipta penerbit agar tidak mengganggu kualitas representasi semantik.
4. Baris 4 memulai iterasi untuk memproses setiap penulis yang terasosiasi dengan artikel ilmiah terkait lengkap dengan urutan kepengarangannya.
5. Baris 5 melakukan ekstraksi ID unik penulis (*Scopus ID*) dan nama lengkap penulis dari kolom teks penulis menggunakan pola ekspresi reguler yang sesuai.
6. Baris 6 memasukkan data identitas penulis yang telah diekstrak ke dalam himpunan data penampung sementara.
7. Baris 7 mencatat hubungan antara artikel ilmiah dan penulis ke dalam tabel penghubung, serta menyimpan urutan kepengarangan penulis yang dimulai dari indeks 1.
8. Baris 8 dan 9 mengakhiri blok perulangan pemrosesan penulis dan pemrosesan artikel.
9. Baris 10 melakukan proses penyimpanan massal (*bulk upsert*) data penulis dari penampung sementara ke dalam tabel authors di basis data guna menghindari duplikasi data.
10. Baris 11 menyimpan seluruh data metadata artikel ilmiah yang telah dibersihkan secara massal ke dalam tabel articles.
11. Baris 12 menyimpan seluruh data relasi kepengarangan beserta informasi urutan penulis secara massal ke dalam tabel articles_authors.

5.2.2 Implementasi Fungsi *Vector Slicing*

Algoritme 2: Database Function <i>sub_vector</i>	
1	INPUT: <i>full_vector</i> , <i>target_dims</i>
2	OUTPUT: <i>normalized_sub_vector</i>
3	Convert <i>full_vector</i> to array
4	IF <i>target_dims</i> > length(<i>full_vector</i>) THEN
5	RAISE EXCEPTION
6	END IF
7	<i>sub_vec</i> = slice(<i>full_vector</i> , 1, <i>target_dims</i>)
8	<i>norm_factor</i> = sqrt(sum(<i>sub_vec</i> ²))
9	FOR each element in <i>sub_vec</i> :
10	element = element / <i>norm_factor</i>
11	END FOR
12	RETURN <i>sub_vec</i>

Modul ini merupakan implementasi fungsi internal pada basis data PostgreSQL untuk mengekstraksi dan menormalisasi sub-vektor dari dimensi penuh ke dimensi target yang lebih rendah sesuai kebutuhan ekstraksi MRL. Penjelasan tahapan program pada Algoritme 2 di atas adalah sebagai berikut:

1. Baris 1 mendefinisikan parameter masukan berupa vektor dimensi penuh (*full_vector*) dan panjang dimensi target yang diinginkan (*target_dims*).
2. Baris 2 mendefinisikan parameter keluaran berupa sub-vektor baru yang telah dinormalisasi menggunakan normalisasi L2.
3. Baris 3 melakukan konversi tipe data vektor masukan menjadi bentuk larik (*array*) agar dapat diproses oleh fungsi internal basis data.
4. Baris 4 memeriksa kondisi apabila dimensi target yang diminta bernilai lebih besar daripada panjang dimensi vektor asli.
5. Baris 5 menghentikan proses eksekusi dan memicu galat (*exception*) apabila kondisi pada baris 4 terpenuhi.
6. Baris 6 mengakhiri blok pemeriksaan kondisi batas dimensi vektor.
7. Baris 7 melakukan pemotongan larik vektor dari indeks pertama hingga indeks dimensi target yang diminta untuk menghasilkan sub-vektor mentah.
8. Baris 8 menghitung faktor normalisasi L2 dengan cara mengakarkan jumlah kuadrat dari seluruh elemen sub-vektor mentah.
9. Baris 9 memulai perulangan untuk memperbaiki nilai setiap elemen di dalam sub-vektor.

10. Baris 10 membagi nilai setiap elemen sub-vektor dengan faktor normalisasi L2 agar dihasilkan sub-vektor baru yang memiliki panjang satuan (*unit length*).
11. Baris 11 mengakhiri proses perulangan normalisasi elemen vektor.
12. Baris 12 mengembalikan sub-vektor yang telah dinormalisasi sebagai hasil keluaran dari fungsi basis data.

5.3 Implementasi Pencarian Pakar

5.3.1 Implementasi Algoritme *Two-Stage Retrieval*

Algoritme 3: Eksekusi <i>Two-Stage Retrieval</i>	
1	INPUT: query_embedding, initial_dims
2	OUTPUT: reranked_docs
3	shortlist = Query indeks HNSW menggunakan sub_vector(query_embedding, initial_dims)
4	IF initial_dims != 768 THEN
5	FOR setiap dokumen di dalam shortlist:
6	doc.score = 1 - cosine_distance(doc.embedding, query_embedding)
7	END FOR
8	reranked_docs = sort_descending(shortlist berdasarkan score)
9	ELSE
10	reranked_docs = shortlist
11	END IF
12	RETURN reranked_docs

Modul ini mengimplementasikan mekanisme pencarian berjenjang dua tahap untuk mempercepat proses penemuan dokumen kandidat tanpa mengorbankan kualitas akhir pemeringkatan semantic. Penjelasan tahapan program pada Algoritme 3 di atas adalah sebagai berikut:

1. Baris 1 mendefinisikan parameter input berupa vektor embedding kueri pengguna (*query_embedding*) dan panjang dimensi awal untuk tahap penyaringan pertama (*initial_dims*).
2. Baris 2 mendefinisikan parameter output berupa daftar pendek dokumen yang telah diurutkan ulang (*reranked_docs*) menggunakan dimensi penuh.
3. Baris 3 menjalankan kueri tahap pertama untuk mencari dokumen tetangga terdekat pada indeks HNSW basis data menggunakan fungsi sub_vector dengan dimensi rendah, sehingga diperoleh daftar pendek (*shortlist*) berisi 100 dokumen kandidat.
4. Baris 4 memeriksa kondisi apakah dimensi awal pencarian yang digunakan bukan merupakan dimensi penuh (768 dimensi).

5. Baris 5 memulai perulangan untuk memproses ulang setiap dokumen kandidat yang ada di dalam daftar pendek apabila kondisi pada baris 4 terpenuhi.
6. Baris 6 menghitung kembali nilai kemiripan kosinus (*cosine similarity*) secara presisi menggunakan representasi dimensi penuh 768 antara embedding dokumen dan embedding kueri.
7. Baris 7 mengakhiri perulangan kalkulasi kemiripan kosinus dimensi penuh untuk dokumen kandidat.
8. Baris 8 mengurutkan ulang daftar pendek dokumen kandidat secara menurun (*descending*) berdasarkan skor kemiripan kosinus dimensi penuh yang baru saja dihitung.
9. Baris 9 mendefinisikan kondisi alternatif apabila dimensi awal yang digunakan sudah merupakan dimensi penuh.
10. Baris 10 menetapkan daftar pendek hasil pencarian tahap pertama langsung sebagai hasil akhir tanpa perlu melakukan proses perhitungan ulang kemiripan kosinus.
11. Baris 11 mengakhiri blok percabangan pengkondisian dimensi.
12. Baris 12 mengembalikan daftar dokumen hasil pengurutan ulang sebagai hasil keluaran modul pencarian.

5.3.2 Implementasi Pemodelan dan Agregasi Skor Kepakaran

Algoritme 4: Pemodelan SDC dan Agregasi CombSUM	
1	INPUT: <code>reranked_articles</code> , <code>author_contributions</code>
2	OUTPUT: <code>top_experts</code>
3	Inisialisasi <code>expert_scores</code> = hash map kosong
4	FOR setiap penulis di dalam <code>reranked_articles</code> :
5	<code>weight</code> = <code>get_sdc_weight(author.order)</code>
6	<code>weighted_score</code> = <code>author.article_similarity * weight</code>
7	<code>expert_scores[author.id] += weighted_score</code>
8	END FOR
9	<code>top_experts</code> = <code>sort_descending(expert_scores)</code>
10	RETURN <code>top_experts</code>

Modul ini mengimplementasikan fungsi agregasi CombSUM yang dimodifikasi menggunakan skema pembobotan posisi penulis (*Linear SDC*) guna menghitung skor kepakaran akhir bagi setiap kandidat pakar. Penjelasan tahapan program pada Algoritme 4 di atas adalah sebagai berikut:

1. Baris 1 menetapkan parameter masukan berupa daftar artikel yang telah diurutkan ulang (*reranked_articles*) dan data kontribusi relasi kepengarangan penulis (*author_contributions*).

2. Baris 2 menetapkan parameter keluaran berupa daftar peringkat pakar teratas (*top_experts*).
3. Baris 3 membuat struktur data tabel pencarian (*hash map*) kosong dengan nama *expert_scores* untuk menampung skor kepakaran sementara bagi setiap penulis.
4. Baris 4 memulai perulangan untuk memproses kontribusi kepakaran penulis dari setiap artikel yang relevan di dalam daftar artikel hasil pencarian.
5. Baris 5 menghitung nilai bobot kontribusi penulis berdasarkan urutan posisi kepengarangan mereka menggunakan rumus peluruhan linier dengan batas minimum (*Linear SDC*).
6. Baris 6 menghitung nilai skor kontribusi artikel berbobot dengan cara mengalikan nilai kemiripan dokumen terhadap kueri dengan nilai bobot posisi penulis.
7. Baris 7 mengakumulasikan skor kontribusi berbobot tersebut ke dalam tabel *expert_scores* berdasarkan ID unik penulis terkait.
8. Baris 8 mengakhiri proses perulangan kalkulasi akumulasi skor kepakaran penulis.
9. Baris 9 mengurutkan daftar tabel *expert_scores* secara menurun berdasarkan total nilai skor kepakaran guna memosisikan pakar paling relevan di peringkat teratas.
10. Baris 10 mengembalikan daftar peringkat pakar yang telah terurut sebagai hasil akhir modul pemodelan kepakaran.

5.4 Implementasi Layanan Web

5.4.1 Implementasi API Backend

Algoritme 5: Pairwise Search Endpoint	
1	INPUT: query_payload
2	OUTPUT: JSON response
3	Validate query_payload
4	prefixed_query = "search_query: " + query_payload
5	query_embedding = call_embedding_api(prefixed_query)
6	ASync run_pairwise_search(query_embedding):
7	Select Model A & Model B randomly
8	Concurrently search experts for Model A & B using query_embedding
9	Measure execution latencies for A & B
10	Save search log to "evaluation_results" table (get evaluation_id)
11	RETURN JSON(evaluation_id, results_a, results_b)

Modul ini diimplementasikan menggunakan kerangka kerja Flask untuk menyediakan *endpoint* API pencarian yang memproses kueri pengguna secara asinkron guna mendukung mekanisme evaluasi *pairwise comparison*. Penjelasan tahapan program pada Algoritme 5 di atas adalah sebagai berikut:

1. Baris 1 mendefinisikan data masukan berupa objek teks kueri pencarian (*query_payload*) yang dikirimkan oleh pengguna melalui aplikasi klien.
2. Baris 2 mendefinisikan data keluaran berupa respons berformat JSON yang berisi hasil pencarian dan log evaluasi kueri.
3. Baris 3 menjalankan modul validasi untuk memastikan bahwa data kueri masukan tidak kosong dan memenuhi syarat keamanan parameter input.
4. Baris 4 menambahkan *prefix* berupa "search_query: " pada kueri mentah guna menyesuaikan kueri dengan spesifikasi instruksi model embedding *nomic-embed*.
5. Baris 5 mengirimkan teks kueri ber-awalan ke layanan inferensi eksternal (TEI Service) guna mendapatkan vektor representasi kueri.
6. Baris 6 mendefinisikan blok eksekusi asinkron (*async*) untuk menjalankan proses pencarian pakar berpasangan.
7. Baris 7 memilih secara acak dua konfigurasi model dimensi Matryoshka yang berbeda sebagai objek pembandingan (Model A dan Model B).
8. Baris 8 menjalankan fungsi pencarian pakar secara bersamaan (*concurrently*) menggunakan kueri embedding yang sama untuk Model A dan Model B.
9. Baris 9 mengukur dan mencatat waktu latensi komputasi eksekusi pencarian secara presisi untuk masing-masing konfigurasi model.
10. Baris 10 menyimpan log hasil pencarian berpasangan beserta catatan latensinya ke dalam tabel *evaluation_results* di basis data relasional serta mengembalikan ID evaluasi (*evaluation_id*).
11. Baris 11 mengembalikan data hasil pencarian dari kedua model beserta identitas ID evaluasi ke antarmuka pengguna dalam format JSON terstruktur.

5.4.2 Implementasi Pencatatan Preferensi

Algoritme 6: Preference Logger	
1	INPUT: <i>evaluation_id</i> , <i>choice</i>
2	Fetch <i>model_a_id</i> , <i>model_b_id</i> using <i>evaluation_id</i>
3	IF <i>choice</i> == 'a' THEN
4	<i>winner_id</i> = <i>model_a_id</i>
5	ELSE IF <i>choice</i> == 'b' THEN
6	<i>winner_id</i> = <i>model_b_id</i>
7	ELSE
8	<i>winner_id</i> = NULL
9	END IF
10	UPDATE "evaluation_results" SET <i>winner_model_id</i> = <i>winner_id</i> , <i>status</i> = True
11	COMMIT database transaction

Modul ini bertanggung jawab untuk mencatat pilihan preferensi juri atau pengguna (apakah hasil pencarian Model A lebih baik, Model B lebih baik, atau keduanya setara) ke dalam basis data berdasarkan ID evaluasi yang terkait. Penjelasan tahapan program pada Algoritme 6 di atas adalah sebagai berikut:

1. Baris 1 menerima data masukan berupa ID unik evaluasi (*evaluation_id*) dan label keputusan pilihan (*choice*).
2. Baris 2 melakukan kueri pengambilan data (*fetch*) ke pangkalan data untuk mengidentifikasi ID model yang digunakan sebagai Model A (*model_a_id*) dan Model B (*model_b_id*) berdasarkan ID evaluasi terkait.
3. Baris 3 memeriksa kondisi apabila pilihan keputusan bernilai 'a', yang mengindikasikan bahwa hasil dari Model A dinilai lebih relevan.
4. Baris 4 menetapkan ID dari Model A (*model_a_id*) sebagai ID pemenang (*winner_id*) jika kondisi pada baris 3 terpenuhi.
5. Baris 5 mendefinisikan kondisi alternatif apabila pilihan keputusan bernilai 'b', yang mengindikasikan bahwa hasil dari Model B dinilai lebih relevan.
6. Baris 6 menetapkan ID dari Model B (*model_b_id*) sebagai ID pemenang (*winner_id*) jika kondisi pada baris 5 terpenuhi.
7. Baris 7 mendefinisikan kondisi pengecualian jika pilihan keputusan tidak bernilai 'a' maupun 'b', yang mengindikasikan bahwa hasil pencarian kedua model dinilai setara (seri).
8. Baris 8 menetapkan ID pemenang (*winner_id*) sebagai nilai kosong (NULL) jika kondisi pengecualian pada baris 7 terpenuhi.
9. Baris 9 mengakhiri seluruh blok pengkondisian logika penentuan pemenang turnamen.

10. Baris 10 menjalankan perintah kueri pembaruan data (*UPDATE*) pada tabel *evaluation_results* di basis data untuk menyimpan ID pemenang serta memperbarui status verifikasi evaluasi menjadi benar (*True*).
11. Baris 11 melakukan operasi komit (*COMMIT*) transaksi pangkalan data guna memastikan bahwa seluruh pembaruan data tersimpan secara permanen dan aman.

5.4.3 Implementasi Antarmuka Pengguna

Algoritme 7: Frontend Preference	
1	INPUT: choice
2	IF evaluation_id is empty THEN
3	RETURN
4	END IF
5	Disable voting buttons in UI
6	POST payload(evaluation_id, choice) to '/preference'
7	IF status == SUCCESS THEN
8	Show toast notification
9	Clear search results and reset evaluation_id
10	ELSE
11	Re-enable voting buttons in UI
12	END IF

Modul ini diimplementasikan pada sisi klien (*frontend*) menggunakan JavaScript untuk mengatur interaksi penekanan tombol preferensi pengguna secara dinamis dan mengirimkannya ke API backend Flask. Penjelasan tahapan program pada Algoritme 7 di atas adalah sebagai berikut:

1. Baris 1 menerima parameter input berupa tindakan klik tombol pilihan preferensi (*choice*) oleh pengguna pada halaman web.
2. Baris 2 memeriksa kondisi apakah variabel ID evaluasi (*evaluation_id*) yang sedang aktif berada dalam keadaan kosong.
3. Baris 3 menghentikan proses eksekusi fungsi jika kondisi ID evaluasi kosong pada baris 2 terpenuhi agar tidak terjadi pengiriman data yang tidak valid.
4. Baris 4 mengakhiri blok pengecekan validitas ID evaluasi.
5. Baris 5 melakukan penonaktifan (*disable*) seluruh tombol pilihan preferensi pada antarmuka pengguna guna mencegah terjadinya pengiriman data ganda secara tidak sengaja oleh pengguna.
6. Baris 6 melakukan pengiriman data muatan berisi ID evaluasi dan label pilihan ke titik akhir layanan backend Flask menggunakan metode HTTP POST menuju jalur URL '/preference'.

7. Baris 7 memeriksa kondisi apabila respons status dari server backend menunjukkan keberhasilan (*SUCCESS*).
8. Baris 8 menampilkan notifikasi melayang (*toast notification*) pada layar pengguna untuk mengonfirmasi bahwa data pilihan preferensi telah berhasil direkam.
9. Baris 9 membersihkan tampilan daftar hasil pencarian dari layar antarmuka serta mengosongkan kembali variabel ID evaluasi untuk mempersiapkan sesi pencarian berikutnya.
10. Baris 10 mendefinisikan kondisi alternatif apabila pengiriman data ke server backend mengalami kegagalan.
11. Baris 11 mengaktifkan kembali (*re-enable*) tombol pilihan preferensi pada antarmuka pengguna agar pengguna dapat mencoba mengirimkan kembali pilihan mereka.
12. Baris 12 mengakhiri seluruh blok logika pemrosesan interaksi preferensi di sisi klien.

5.5 Implementasi *Pipeline* Evaluasi Otomatis

5.5.1 Implementasi Generator Kueri Sintetis

Algoritme 8: Synthetic Query Generation	
1	Algoritme 8: Synthetic Query Generation
2	INPUT: Database articles
3	OUTPUT: all generated queries
4	anchor_papers = SELECT top 100 articles ORDER BY cited_by DESC
5	FOR each paper in anchor_papers:
6	prompt = construct_prompt(paper.title, paper.abstract)
7	response = call_llm_api(prompt)
8	query_text = clean_text(response)
9	Save query_text linked to paper.eid
10	END FOR
11	RETURN all generated queries

Modul ini bertanggung jawab untuk memproses artikel-artikel ilmiah dengan sitasi tertinggi dari basis data guna membangkitkan kueri pencarian alami secara sintetis menggunakan bantuan API LLM Gemma. Penjelasan tahapan program pada Algoritme 8 di atas adalah sebagai berikut:

1. Baris 1 mendefinisikan nama modul yaitu proses pembuatan kueri secara sintetis (*Synthetic Query Generation*).
2. Baris 2 menetapkan basis data artikel ilmiah (*articles*) sebagai sumber data masukan utama.

3. Baris 3 menetapkan variabel penampung seluruh kueri yang berhasil dibuat (*all_generated_queries*) sebagai hasil keluaran.
4. Baris 4 menjalankan perintah kueri seleksi SQL untuk mengambil data 100 artikel ilmiah teratas yang diurutkan berdasarkan jumlah sitasi terbanyak (*cited_by DESC*) sebagai representasi.
5. Baris 5 memulai perulangan untuk memproses setiap dokumen artikel ilmiah yang terpilih di dalam daftar representasi.
6. Baris 6 menyusun *prompt* terstruktur dengan menggabungkan informasi judul (*title*) dan abstrak (*abstract*) dari artikel representasi terkait.
7. Baris 7 mengirimkan teks perintah tersebut menuju layanan API LLM guna mendapatkan respons teks kueri yang relevan.
8. Baris 8 menjalankan fungsi pembersihan teks (*clean_text*) pada respons LLM untuk menghilangkan karakter asing dan format spasi ganda.
9. Baris 9 menyimpan teks kueri sintetis baru ke dalam basis data dengan menghubungkannya secara langsung ke ID unik artikel representasi (*paper.eid*) sebagai data *ground truth*.
10. Baris 10 mengakhiri proses perulangan pembuatan kueri untuk seluruh artikel representasi.
11. Baris 11 mengembalikan daftar kueri sintetis yang telah terkumpul sebagai hasil akhir modul *query generator*.

5.5.2 Implementasi Juri LLM

Algoritme 9: LLM Judge Evaluator	
1	INPUT: query, results_a, results_b
2	OUTPUT: judge_decision
3	formatted_a = format_experts_and_abstracts(results_a)
4	formatted_b = format_experts_and_abstracts(results_b)
5	prompt = construct_judge_prompt(query, formatted_a, formatted_b)
6	config = GenerateConfig(response_type=JSON, temp=0.1)
7	response = call_llm_api(prompt, config)
8	judge_decision = parse_json(response)
9	RETURN judge_decision

Modul ini mengimplementasikan teknik evaluasi otomatis berbasis *LLM-as-a-Judge* menggunakan API Gemma untuk menentukan pemenang perbandingan hasil pencarian secara objektif. Penjelasan tahapan program pada Algoritme 9 di atas adalah sebagai berikut:

1. Baris 1 menerima tiga parameter masukan utama yang terdiri atas teks kueri, daftar hasil pencarian pakar dari Model A (*results_a*), dan daftar hasil dari Model B (*results_b*).
2. Baris 2 mendefinisikan parameter hasil keluaran berupa keputusan juri otomatis (*judge_decision*).
3. Baris 3 mengubah format daftar pakar beserta metadata dokumen pendukung dari Model A menjadi bentuk teks naratif terstruktur (*formatted_a*) agar mudah dipahami oleh LLM.
4. Baris 4 mengubah format daftar pakar beserta metadata dokumen pendukung dari Model B menjadi bentuk teks naratif terstruktur (*formatted_b*).
5. Baris 5 menyusun teks perintah evaluasi (*prompt*) yang menginstruksikan LLM untuk bertindak sebagai juri netral dan membandingkan relevansi hasil dari kedua model berdasarkan kemiripan semantik.
6. Baris 6 menetapkan konfigurasi generator LLM dengan mengatur format respons keluaran berupa JSON (*response_type=JSON*) dan tingkat kreativitas rendah (*temperature=0.1*) guna menjaga konsistensi keputusan.
7. Baris 7 memanggil API LLM Gemini dengan mengirimkan teks perintah evaluasi beserta objek konfigurasi yang telah ditentukan.
8. Baris 8 melakukan penguraian (*parsing*) terhadap respons JSON dari LLM untuk mengekstrak data keputusan pemenang beserta alasan analisisnya.
9. Baris 9 mengembalikan data keputusan hasil penilaian juri otomatis sebagai keluaran akhir dari modul evaluator.

5.5.3 Implementasi Pencatatan Preferensi Juri LLM

Algoritme 10: LLM Judge Logger	
1	INPUT: results, decision, reasoning, latencies
2	query_id = Insert query text and vector into DB
3	results_identical = (List A == List B)
4	winner_id = map_choice_to_id(decision)
5	Insert evaluation record (query_id, winner_id, latencies, reasoning, results_identical)
6	Bulk insert expert lists linked to evaluation_id
7	COMMIT database transaction

Algoritma ini bertanggung jawab untuk mencatat hasil keputusan juri otomatis secara massal ke dalam pangkalan data relasional serta memperbarui skor Elo untuk setiap model yang terlibat secara berkala. Penjelasan tahapan program pada Algoritme 10 di atas adalah sebagai berikut:

1. Baris 1 menetapkan data masukan yang mencakup daftar hasil pencarian (*results*), keputusan pemenang (*decision*), argumen alasan juri (*reasoning*), dan catatan latensi pencarian vektor (*latencies*).
2. Baris 2 menjalankan fungsi penyimpanan untuk memasukkan data teks kueri beserta representasi vektor kueri ke dalam tabel queries di basis data dan mengembalikan ID kueri (*query_id*).
3. Baris 3 melakukan pemeriksaan logika untuk menentukan apakah daftar hasil pencarian dari Model A dan Model B menghasilkan urutan pakar yang identik (*results_identical*).
4. Baris 4 menjalankan fungsi pemetaan (*map_choice_to_id*) untuk mengonversi label keputusan tekstual juri menjadi bentuk ID model pemenang yang sesuai di pangkalan data.
5. Baris 5 melakukan operasi penyimpanan data evaluasi baru ke dalam tabel evaluation_results dengan mencatat ID kueri, ID model pemenang, catatan waktu latensi dari kedua model, argumen analisis juri, serta status kesamaan hasil.
6. Baris 6 melakukan penyimpanan massal (*bulk insert*) daftar hasil pakar yang direkomendasikan oleh kedua model ke dalam tabel expert_results dengan menghubungkannya ke ID evaluasi terkait.
7. Baris 7 melakukan komit (*COMMIT*) transaksi pangkalan data untuk memastikan seluruh data transaksi evaluasi otomatis berhasil disimpan secara permanen dan konsisten di dalam basis data.

BAB 6 HASIL DAN PEMBAHASAN

6.1 Hasil Eksperimen

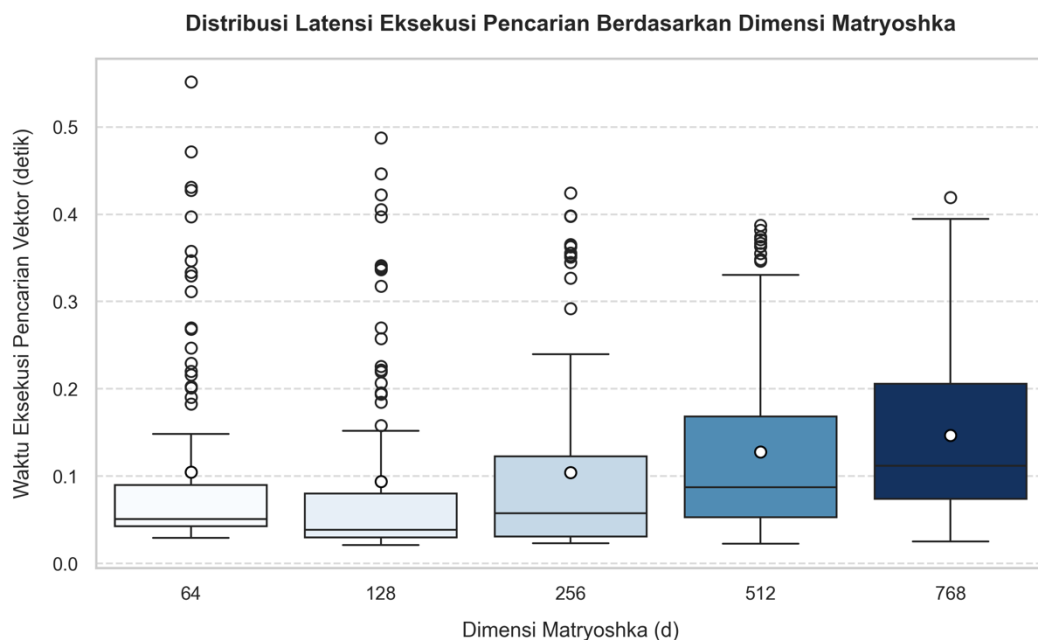
6.1.1 Hasil Pengujian Latensi Vektor

Pengukuran waktu pencarian vektor dilakukan terhadap 100 kueri pada lima variasi dimensi Matryoshka. Ringkasan parameter statistik deskriptif beserta persentase perbedaan latensi terhadap dimensi *baseline* disajikan secara lengkap pada Tabel 6.1.

Tabel 6.1 Tabel Hasil Pengujian Latensi Vektor

Dimensi	Mean (Detik)	Persentase Perbedaan	Std	Minimum (Detik)	Median (Detik)	Maksimum (Detik)
64	0,104488	-28,70%	0,114281	0,029263	0,050827	0,551721
128	0,093755	-36,02%	0,113696	0,021191	0,038455	0,487365
256	0,103911	-29,09%	0,108519	0,023089	0,057443	0,424463
512	0,127612	-12,92%	0,103049	0,022729	0,087025	0,387552
768	0,146547	Baseline (0,00%)	0,100842	0,025079	0,112059	0,419222

Gambar 6.1 Distribusi Latensi Pencarian Vektor Berdasarkan Dimensi *Matryoshka*



Visualisasi distribusi sebaran latensi dari kelima kelompok dimensi di atas menunjukkan bahwa pemangkasan dimensi secara konsisten mempercepat batas bawah waktu eksekusi pencarian. Distribusi sebaran data latensi ini disajikan secara visual pada Gambar 6.1.

6.1.2 Hasil Pengujian Signifikansi Latensi Pencarian Vektor

Analisis *One-way ANOVA* diterapkan untuk mendeteksi perbedaan rata-rata latensi antar-dimensi secara global. Pengujian tersebut menghasilkan nilai *F-Statistic* sebesar 3,9255 dengan *p-value* sebesar 0,0037. Karena nilai *p-value* berada di bawah ambang batas signifikansi 0,05, hipotesis nol ditolak sehingga analisis dilanjutkan dengan uji *post-hoc Tukey HSD* untuk memetakan perbedaan signifikansi antar-pasangan dimensi secara spesifik sebagaimana disajikan pada Tabel 6.2.

Tabel 6.2 Tabel Hasil Pengujian *Tukey HSD*

Pasangan Dimensi	<i>Meandiff</i>	p-adj	Batas Bawah (95% CI)	Batas Atas (95% CI)	Tolak H0 (<i>Reject</i>)
64 vs 128	-0,0107	0,9562	-0,0526	0,0312	False
64 vs 256	-0,0006	1,0000	-0,0425	0,0413	False
64 vs 512	0,0231	0,5558	-0,0188	0,0650	False
64 vs 768	0,0421	0,0486	0,0002	0,0840	True
128 vs 256	0,0102	0,9640	-0,0317	0,0521	False
128 vs 512	0,0339	0,1768	-0,0080	0,0758	False
128 vs 768	0,0528	0,0055	0,0109	0,0947	True
256 vs 512	0,0237	0,5312	-0,0182	0,0656	False
256 vs 768	0,0426	0,0438	0,0007	0,0845	True
512 vs 768	0,0189	0,7295	-0,0230	0,0608	False

6.1.3 Hasil Pengujian Latensi Vektor

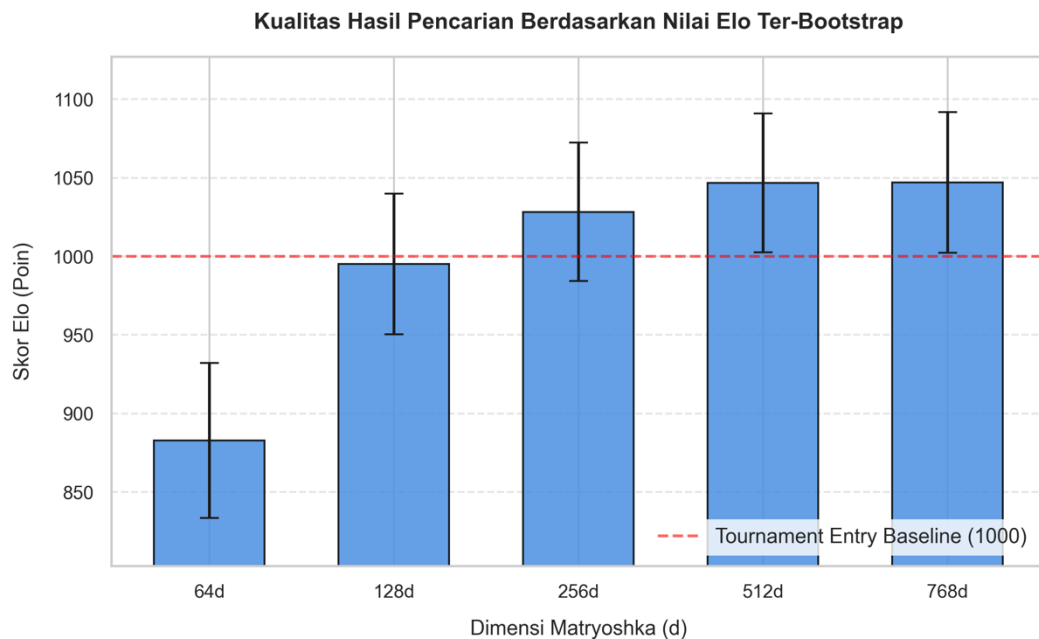
Pengukuran kualitas hasil pencarian pakar dilakukan menggunakan metode *blind pairwise comparison* dengan juri LLM, yang kemudian dikonversi menjadi skor Elo. Estimasi kestabilan parameter kualitas dihitung melalui simulasi *Bootstrap Monte Carlo* sebanyak 1.000 iterasi. Ringkasan hasil estimasi skor Elo beserta persentase perbedaannya terhadap dimensi *baseline* disajikan pada Tabel 6.3.

Tabel 6.3 Tabel Estimasi Kualitas Dimensi dengan *Bootstrap Elo Rating*

Dimensi	Rata-rata ELO	Persentase Perbedaan	Standar Error	Batas Bawah ELO (95% CI)	Batas Atas ELO (95% CI)
64	882,86	-15,68%	49,279120	788,25	980,68
128	995,04	-4,97%	44,747087	912,98	1087,32
256	1028,27	-1,79%	44,034339	947,32	1111,23
512	1046,74	-0,03%	44,259348	963,13	1133,11
768	1047,06	<i>Baseline</i> (0,00%)	44,811896	952,73	1129,76

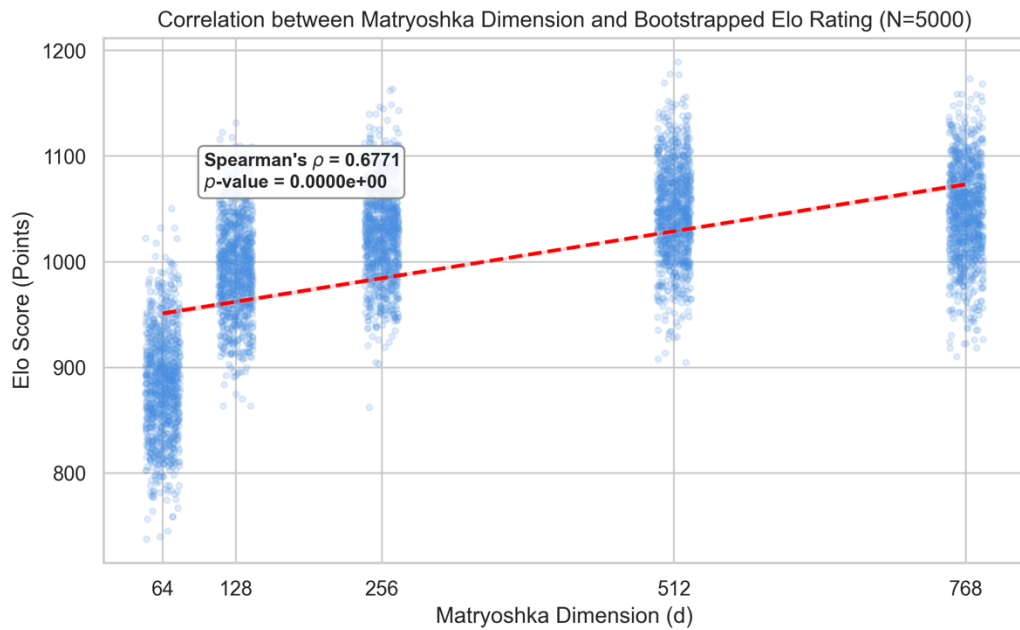
Perbandingan kualitas hasil pencarian masing-masing dimensi terhadap *tournament entry baseline* yang sebesar 1000 Elo divisualisasikan pada Gambar 6.2.

Gambar 6.2 Kualitas Hasil Pencarian Berddasarkan Nilai *Bootstrap Elo Rating*



Uji *Spearman's rank correlation* yang dijalankan pada tingkat kepercayaan 95 persen menghasilkan koefisien korelasi rho sebesar 0,6771 dengan p-value sebesar 0,0000. Visualisasi sebaran hubungan korelasi ini disajikan pada Gambar 6.3.

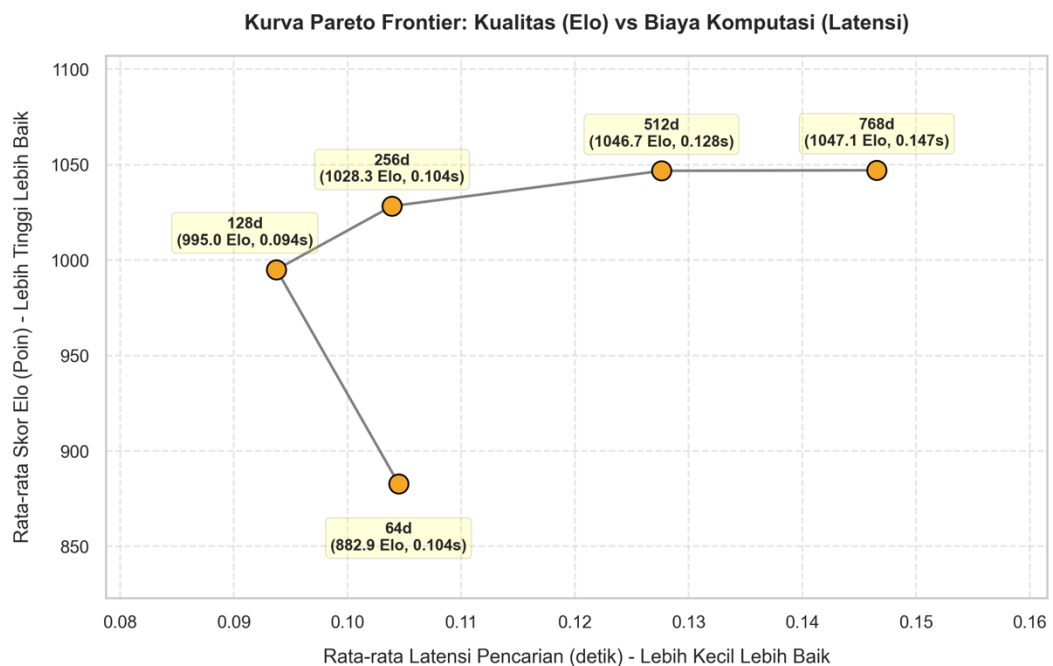
Gambar 6.3 Korelasi Antara Dimensi *Matryoshka* dan *Bootstrap Elo Rating*



6.1.4 Hasil Pemetaan *Pareto Frontier*

Hubungan *trade-off* antara rata-rata latensi pencarian dan rata-rata skor Elo dipetakan menggunakan analisis optimasi multiobjektif. Peta ruang pencarian ini divisualisasikan melalui kurva *Pareto frontier* pada Gambar 6.4.

Gambar 6.4 Kurva *Pareto Frontier*



6.2 Analisa Hasil

6.2.1 Pengaruh Pemangkasan Dimensi terhadap Kinerja Sistem

Tujuan utama dari penelitian ini adalah merancang sistem pencarian pakar berbasis *Document-based Model* yang mampu mengoptimalkan *trade-off* antara kecepatan komputasi dan kualitas hasil menggunakan teknik MRL. Merujuk pada hasil eksperimen, prototipe sistem *two-stage retrieval* yang dibangun telah berfungsi sesuai dengan tujuan perancangannya.

Sistem berhasil memotong latensi pencarian secara nyata dengan menerapkan pemangkasan vektor MRL pada *shortlisting* sehingga beban komputasi pada basis data PostgreSQL menjadi lebih ringan. Setelah itu, akurasi pemeringkatan dipertahankan menggunakan representasi dimensi penuh pada tahap *reranking* sehingga kualitas pencarian tetap terjaga secara optimal.

6.2.2 Analisis *Trade-off* Kecepatan dan Kualitas

Berdasarkan aspek efisiensi latensi, pengurangan dimensi terbukti secara nyata memotong waktu eksekusi pencarian vektor pada sistem. Pemangkasan dimensi menjadi 128 menghasilkan efisiensi latensi tertinggi dengan waktu eksekusi rata-rata 0,093 detik, sehingga sistem berjalan 36,02 persen lebih cepat dibandingkan dengan dimensi penuh 768. Efisiensi ini terjadi karena operasi perkalian titik pada dimensi rendah membutuhkan operasi komputasi yang jauh lebih sedikit pada memori pangkalan data. Peningkatan kecepatan yang hampir setara juga dicatatkan oleh dimensi 256 yang mampu mengeksekusi pencarian 29,09 persen lebih cepat daripada dimensi acuan.

Namun, pengujian ini juga menyingkap adanya anomali pada dimensi 64. Secara teoritis, dimensi yang lebih kecil seharusnya menghasilkan waktu eksekusi yang lebih cepat karena beban komputasinya lebih ringan. Akan tetapi, data pada Tabel 6.1 memperlihatkan bahwa dimensi 64 justru mencatatkan rata-rata latensi sebesar 0,104 detik, yang berarti lebih lambat sebesar 11,45 persen dibandingkan dengan dimensi 128 yang memiliki dimensi lebih besar. Anomali ini terjadi karena struktur data berdimensi sangat rendah cenderung mengalami penurunan tingkat selektivitas kueri pada indeks HNSW. Akibatnya, sistem harus melakukan pemindaian ekstra untuk menyaring kandidat dokumen yang kurang relevan pada memori fisik, sehingga efisiensi waktu komputasi yang diharapkan dari pemangkasan dimensi menjadi tidak optimal.

Konsekuensi dari peningkatan efisiensi tersebut adalah terjadinya penurunan kualitas representasi informasi semantik karena hilangnya beberapa dimensi penjelas. Meskipun demikian, pengurangan dimensi terbukti tidak selalu mengorbankan akurasi secara drastis apabila batas pemangkasan dilakukan secara terukur. Pemangkasan dimensi menjadi 128 hanya mengakibatkan penurunan kualitas sebesar 4,97 persen dengan perolehan skor Elo sebesar 995,05 poin. Penurunan yang sangat tipis juga dicatatkan oleh dimensi 256 yang hanya mengalami penurunan kualitas sebesar 1,79 persen dari dimensi acuan penuh. Sebaliknya, pemangkasan ekstrem pada dimensi 64 terbukti merugikan karena

menurunkan kualitas secara drastis hingga mencapai 15,68 persen sehingga skor Elo jatuh di bawah batas dasar turnamen.

Di sisi lain, analisis terhadap dimensi yang lebih besar menunjukkan adanya gejala hasil yang melandai (*diminishing returns*) pada dimensi 512 dan 768. Selisih skor Elo antara dimensi 512 (1046,75 poin) dan dimensi penuh 768 (1047,07 poin) tercatat sangat tipis, yaitu hanya sebesar 0,03 persen. Meskipun perbedaan kualitas ini hampir tidak terasa dalam hasil pencarian aktual, konsekuensi biaya latensi yang harus dibayar sangat mahal. Dimensi penuh 768 membutuhkan rata-rata waktu 0,146 detik, sedangkan dimensi 512 mengeksekusi pencarian dalam waktu 0,127 detik. Hal ini menunjukkan bahwa penambahan dimensi di atas 256 tidak lagi memberikan kontribusi peningkatan akurasi yang berarti, tetapi justru membebani sumber daya komputasi sistem secara signifikan.

Melalui pemetaan batas efisiensi Pareto pada Gambar 6.4, dimensi 128 diidentifikasi sebagai konfigurasi kompromi yang paling optimal untuk penerapan praktis. Konfigurasi ini direkomendasikan karena mampu mempercepat latensi pencarian secara maksimal hingga menghemat waktu sebesar 36,02 persen sembari menghindari anomali latensi dimensi 64, serta mampu mempertahankan kualitas di dekat batas dasar turnamen dengan toleransi kehilangan akurasi hanya sebesar 4,97 persen. Di sisi lain, dimensi 256 dapat dipilih sebagai konfigurasi alternatif apabila sistem membutuhkan akurasi yang lebih tinggi karena hanya kehilangan 1,79 persen kualitas sembari tetap memberikan penghematan waktu pencarian sebesar 29,09 persen tanpa terjebak pada ketidakefisienan komputasi dimensi 512 dan 768.

BAB 7 PENUTUP

7.1 Kesimpulan

Beberapa kesimpulan yang dapat ditarik dari hasil analisis dan pembahasan pada penelitian ini adalah sebagai berikut:

1. Variasi panjang dimensi embedding Matryoshka terbukti secara statistik berpengaruh nyata terhadap latensi waktu eksekusi pencarian vektor pada sistem pencarian pakar. Pemangkasan dimensi dari 768 menjadi 128 mampu memotong latensi komputasi secara signifikan sebesar 36,02 persen karena menurunkan kompleksitas operasi perkalian titik pada pangkalan data, sekaligus menghindari anomali degradasi latensi yang terjadi pada dimensi 64.
2. Variasi panjang dimensi embedding Matryoshka memiliki dampak korelasi positif yang kuat terhadap kualitas hasil pencarian pakar. Meskipun dimensi terkecil menghasilkan penurunan kualitas sebesar 15,68 persen di bawah batas dasar turnamen, dimensi 128 mampu menjaga kualitas hasil pencarian tetap optimal dengan hanya mengalami penurunan akurasi sebesar 4,97 persen dari model acuan penuh, di mana penambahan dimensi di atas 256 terbukti menghasilkan *diminishing returns*.

7.2 Saran

Beberapa saran yang diajukan untuk pengembangan penelitian selanjutnya adalah sebagai berikut:

1. Penelitian selanjutnya disarankan untuk menguji metode MRL ini pada pangkalan data dengan skala dokumen yang lebih besar guna memverifikasi kestabilan latensi serta efisiensi indeks HNSW pada kondisi industri nyata.
2. Eksplorasi penggunaan teknik kuantisasi vektor pada dimensi rendah hasil pangkasan Matryoshka dapat dilakukan pada penelitian berikutnya agar diperoleh penurunan kebutuhan memori penyimpanan pangkalan data yang lebih ekstrem.

DAFTAR REFERENSI

- Bahadori, M., Yaghoubi, M., Haghgoshyie, E., Ghasemi, M. dan Hasanpoor, E., 2020. Patients' and physicians' perspectives and experiences on the quality of medical consultations: a qualitative study. *International Journal of Evidence-Based Healthcare*, 18(2), hlm.247–255. <https://doi.org/10.1097/XEB.0000000000000210>.
- Balog, K., Fang, Y., Rijke, M. de, Serdyukov, P. dan Si, L., 2012. Expertise Retrieval. *Foundations and Trends in Information Retrieval*. [daring] <https://doi.org/10.1561/15000000024>.
- Bengio, Y., Ducharme, R., Vincent, P. dan Jauvin, C., 2003. A Neural Probabilistic Language Model.
- Berry, M.W., Drmac, Z. dan Jessup, E.R., 1999. Matrices, Vector Spaces, and Information Retrieval. *SIAM Review*, 41(2), hlm.335–362. <https://doi.org/10.1137/S0036144598347035>.
- Chaudhary, A., Raman, K. dan Bendersky, M., 2024. It's All Relative! – A Synthetic Query Generation Approach for Improving Zero-Shot Relevance Prediction. Dalam: K. Duh, H. Gomez dan S. Bethard, ed. *Findings of the Association for Computational Linguistics: NAACL 2024*. [daring] Findings 2024. Mexico City, Mexico: Association for Computational Linguistics. hlm.1645–1664. <https://doi.org/10.18653/v1/2024.findings-naacl.107>.
- Chiang, W.-L., Zheng, L., Sheng, Y., Angelopoulos, A.N., Li, T., Li, D., Zhu, B., Zhang, H., Jordan, M., Gonzalez, J.E. dan Stoica, I., 2024. Chatbot Arena: An Open Platform for Evaluating LLMs by Human Preference. Dalam: R. Salakhutdinov, Z. Kolter, K. Heller, A. Weller, N. Oliver, J. Scarlett dan F. Berkenkamp, ed. *Proceedings of the 41st International Conference on Machine Learning*, Proceedings of Machine Learning Research. [daring] PMLR. hlm.8359–8388. Tersedia pada: <<https://proceedings.mlr.press/v235/chiang24b.html>>.
- Cifariello, P., Ferragina, P. dan Ponza, M., 2019. Wiser: A semantic approach for expert finding in academia based on entity linking. *Information Systems*, 82, hlm.1–16. <https://doi.org/10.1016/j.is.2018.12.003>.
- Deng, H., Lyu, M.R. dan King, I., 2009. A generalized Co-HITS algorithm and its application to bipartite graphs. Dalam: *Proceedings of the 15th ACM SIGKDD international conference on Knowledge discovery and data mining*. [daring] KDD09: The 15th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining. Paris France: ACM. hlm.239–248. <https://doi.org/10.1145/1557019.1557051>.

Devlin, J., Chang, M.-W., Lee, K. dan Toutanova, K., 2019. BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding. Dalam: J. Burstein, C. Doran dan T. Solorio, ed. *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 1 (Long and Short Papers)*. [daring] NAACL-HLT 2019. Minneapolis, Minnesota: Association for Computational Linguistics. hlm.4171–4186. <https://doi.org/10.18653/v1/N19-1423>.

Elo, A.E., 1966. *The USCF Rating System: Its Development, Theory, and Applications*. [daring] United States Chess Federation. Tersedia pada: <<https://books.google.com.sg/books?id=onUazQEACAAJ>>.

Gollapalli, S.D., Mitra, P. dan Giles, C.L., 2013. Ranking experts using author-document-topic graphs. Dalam: *Proceedings of the 13th ACM/IEEE-CS joint conference on Digital libraries*. [daring] JCDL '13: 13th ACM/IEEE-CS Joint Conference on Digital Libraries. Indianapolis Indiana USA: ACM. hlm.87–96. <https://doi.org/10.1145/2467696.2467707>.

Gospodinov, M., MacAvaney, S. dan Macdonald, C., 2023. Doc2Query–: When Less is More. Dalam: *Advances in Information Retrieval: 45th European Conference on Information Retrieval, ECIR 2023, Dublin, Ireland, April 2–6, 2023, Proceedings, Part II*. [daring] Berlin, Heidelberg: Springer-Verlag. hlm.414–422. https://doi.org/10.1007/978-3-031-28238-6_31.

Hagen, N.T., 2010. Harmonic publication and citation counting: sharing authorship credit equitably – not equally, geometrically or arithmetically. *Scientometrics*, 84(3), hlm.785–793. <https://doi.org/10.1007/s11192-009-0129-4>.

Huang, L., Yu, W., Ma, W., Zhong, W., Feng, Z., Wang, H., Chen, Q., Peng, W., Feng, X., Qin, B. dan Liu, T., 2025. A Survey on Hallucination in Large Language Models: Principles, Taxonomy, Challenges, and Open Questions. *ACM Transactions on Information Systems*. [daring] <https://doi.org/10.1145/3703155>.

Husain, O., Salim, N., Alias, R.A., Abdelsalam, S. dan Hassan, A., 2019. Expert Finding Systems: A Systematic Review. *Applied Sciences*, 9(20), hlm.4250. <https://doi.org/10.3390/app9204250>.

IBM, 2025. *Apa itu Large Language Models (LLM)?* [daring] Tersedia pada: <<https://www.ibm.com/id-id/think/topics/large-language-models>> [Diakses 22 Agustus 2025].

Kang, Y.-B., Du, H., Forkan, A.R.M., Jayaraman, P.P., Aryani, A. dan Sellis, T., 2023. ExpFinder: A hybrid model for expert finding from text-based expertise data. *Expert Systems with Applications*, 211, hlm.118691. <https://doi.org/10.1016/j.eswa.2022.118691>.

Kusupati, A., Bhatt, G., Rege, A., Wallingford, M., Sinha, A., Ramanujan, V., Howard-Snyder, W., Chen, K., Kakade, S., Jain, P. dan Farhadi, A., 2022. Matryoshka Representation Learning. *Advances in Neural Information Processing Systems*, 35, hlm.30233–30249.

Lighthall, G.K. dan Vazquez-Guillamet, C., 2015. Understanding Decision Making in Critical Care. *Clinical Medicine & Research*, 13(3–4), hlm.156–168. <https://doi.org/10.3121/cmr.2015.1289>.

Malmqvist, L., 2025. Sycophancy in Large Language Models: Causes and Mitigations. Dalam: K. Arai, ed. *Intelligent Computing*. Cham: Springer Nature Switzerland. hlm.61–74. https://doi.org/10.1007/978-3-031-92611-2_5.

Mikolov, T., Sutskever, I., Chen, K., Corrado, G.S. dan Dean, J., 2013. Distributed Representations of Words and Phrases and their Compositionality. Dalam: C.J. Burges, L. Bottou, M. Welling, Z. Ghahramani dan K.Q. Weinberger, ed. *Advances in Neural Information Processing Systems*. [daring] Curran Associates, Inc. Tersedia pada:
<https://proceedings.neurips.cc/paper_files/paper/2013/file/9aa42b31882ec039965f3c4923ce901b-Paper.pdf>.

Moreira, C., Martins, B. dan Calado, P., 2020. *Using Rank Aggregation for Expert Search in Academic Digital Libraries*. <https://doi.org/10.48550/arXiv.1501.05140>.

Nogueira, R. dan Cho, K., 2020. *Passage Re-ranking with BERT*. <https://doi.org/10.48550/arXiv.1901.04085>.

Pennington, J., Socher, R. dan Manning, C., 2014. Glove: Global Vectors for Word Representation. Dalam: *Proceedings of the 2014 Conference on Empirical Methods in Natural Language Processing (EMNLP)*. [daring] Proceedings of the 2014 Conference on Empirical Methods in Natural Language Processing (EMNLP). Doha, Qatar: Association for Computational Linguistics. hlm.1532–1543. <https://doi.org/10.3115/v1/D14-1162>.

Rampisela, T.V. dan Yulianti, E., 2020a. Academic Expert Finding in Indonesia using Word Embedding and Document Embedding: A Case Study of Fasilkom UI. Dalam: *2020 8th International Conference on Information and Communication Technology (ICoICT)*. [daring] 2020 8th International Conference on Information and Communication Technology (ICoICT). hlm.1–6. <https://doi.org/10.1109/ICoICT49345.2020.9166249>.

Rampisela, T.V. dan Yulianti, E., 2020b. Semantic-Based Query Expansion for Academic Expert Finding. Dalam: *2020 International Conference on Asian Language Processing (IALP)*. [daring] 2020 International Conference on Asian Language Processing (IALP). hlm.34–39. <https://doi.org/10.1109/IALP51396.2020.9310492>.

Reimers, N. dan Gurevych, I., 2019. Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks. Dalam: K. Inui, J. Jiang, V. Ng dan X. Wan, ed. *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing (EMNLP-IJCNLP)*. [daring] EMNLP-IJCNLP 2019. Hong Kong, China: Association for Computational Linguistics. hlm.3982–3992. <https://doi.org/10.18653/v1/D19-1410>.

Schall, D., 2015. *Social Network-Based Recommender Systems*. [daring] Cham: Springer International Publishing. <https://doi.org/10.1007/978-3-319-22735-1>.

Singh, S., 2025. ChatGPT Statistics (2025) - Daily & Monthly Active Users. *DemandSage*. Tersedia pada: <<https://www.demandsage.com/chatgpt-statistics/>> [Diakses 22 Agustus 2025].

Wang, L., Lin, J. dan Metzler, D., 2011. A cascade ranking model for efficient ranked retrieval. Dalam: *Proceedings of the 34th international ACM SIGIR conference on Research and development in Information Retrieval, SIGIR '11*. [daring] New York, NY, USA: Association for Computing Machinery. hlm.105–114. <https://doi.org/10.1145/2009916.2009934>.

Zheng, L., Chiang, W.-L., Sheng, Y., Zhuang, S., Wu, Z., Zhuang, Y., Lin, Z., Li, Z., Li, D., Xing, E., Zhang, H., Gonzalez, J. dan Stoica, I., 2023. Judging LLM-as-a-Judge with MT-Bench and Chatbot Arena. *Advances in Neural Information Processing Systems*, 36, hlm.46595–46623.